PrismaQuant and PrismaSCOUT: Operational Rate–Distortion for Mixed-Precision Large Language Model Quantization

Robert Tand
Independent Researcher
robert.tand@icloud.com

May 2026

Abstract

We describe PrismaQuant, a mixed-precision quantization pipeline for large language models, and PrismaSCOUT, the search-and-select algorithm at its core. Mixed-precision allocation is conventionally posed as a constrained discrete optimization in which a per-Linear surrogate cost $\sum_i u_i(f_i)$ is minimized over format assignments under a total-bit budget. We argue that this formulation is structurally biased on modern transformers: the true joint quantization error is non-additive across layers, and surrogate ranks reliably overshoot when many flips are committed at once. We therefore decouple *candidate generation* from *candidate selection*. Surrogate costs propose discrete allocations cheaply via a multi-level hierarchy (L_1 probe, L_2 perturbed-activation fixed point, L_3 propagated end-KL); every assignment that may ship is then gated by a real, end-to-end Kullback–Leibler divergence measured on a held-out calibration split. Selection is performed by a validated-frontier kneedle: the bisection samples (bpp, KL) pairs adaptively, the dominance filter is widened by an explicit KL noise floor, and the chosen knee is verified by a leave-one-out stability diagnostic. A monotone coordinate-descent polish guarantees the shipped assignment is no worse than the chosen frontier point. On Qwen3.6-27B, the pipeline produces a 5.31 bits-per-parameter artifact with held-out KL 0.0151, replacing a previously shipped 5.50 bpp artifact at KL 0.0475 from the same source weights and the same export-time quantization tricks — 11% smaller and 68% lower KL on the same calibration. The artifact is published with DOI `10.57967/hf/8656` [1]. A subsequent *production-faithful* polish (Section 3.8), in which the polish-time evaluator is upgraded to use the same per-Linear quantized weights the exporter ultimately ships — joint-NVFP4 sibling-coherent input global scales, GPTQ reconstruction, scale-sweep, and calibrated activation clip — and the move set is restructured to start from a low-bpp floor with a small bits-budget creep, drives the held-out KL further down to 0.0054 at 5.39 bpp, roughly 2.8× lower KL than the shipped PrismaSCOUT artifact at comparable bpp. We also document, in a separate methodology section, several numerical methods that were proposed and either implemented-but-deprecated or rejected outright during the design process: Lagrangian λ-bisection on the dual gradient, sandwich proximal recalibration, block-DP over architectural cliques, sparse pairwise QUBO refinement, top-K Hessian-eigenvalue covering, surrogate-only knee selection, and a single-anchor probe-only knee predictor. We give the principled argument for each, and the empirical or structural reason it did not become the shipping path.

1 Introduction

Post-training quantization has converged on two dominant axes. Per-tensor methods — GPTQ [4], AWQ [5], AutoRound [6], HALO/HQQ-style rotation preprocessors, and most recently ParoQuant [15] — improve the per-Linear quantization error of a fixed format. Allocation methods — HAWQ-V3 [8], the cross-layer-dependency-aware integer-quadratic-programming

approach of CLADO [9], the cooperative-game CoopQ allocator [10], and the AutoML-style AMQ [11] — choose how many bits to spend where. The two axes are largely orthogonal: an allocator can sit on top of any per-tensor toolkit and decide which Linears get 16, 8, 4, 3, or 2 bits.

The allocation literature is unified by a recurring observation: per-layer sensitivity surrogates (diagonal Hessian traces, Fisher MSEs, output-MSEs) are biased estimators of joint quantization error. The standard remedy is to model the bias — pairwise integer-quadratic programming over an explicit cross-layer dependency model in CLADO [9], second-order ILP in HAWQ-V3 [8], Shapley-based cooperative-game allocation in CoopQ [10]. We take a different tack. *Surrogates generate, real measurement selects.* An allocator does not need a perfect cost model if every candidate it proposes can be cheaply re-scored end-to-end on a held-out calibration split. Cross-layer interactions and propagated effects then cease to be quantities one must *model*; they are quantities one *observes*, automatically, every time a candidate is gated.

This paper reports the design and engineering of a system, named PrismaQuant, that operationalizes that principle, and the search algorithm at its core, PrismaSCOUT (**S**urrogate-**C**ascaded **O**ptimization **U**nder **T**radeoff). Three contributions are new:

1. A **multi-level cost hierarchy** (L_1, L_2, L_3) that separates cheap proposal from increasingly faithful measurement. L_2 is a perturbed-activation fixed point that folds inter-layer coupling into the activation cache rather than into a closed-form interaction term. L_3 is a propagated end-KL measurement performed against a target-specific BF16 paired baseline, lane-batched and tail-amortized so that one L_3 pass at Qwen-27B scale takes a tractable wall time.
2. A **validated-frontier kneedle**: the elbow of the operational rate–distortion curve is found on *measured* (bpp, KL) pairs, not on a surrogate hull. We add a KL noise floor to the dominance filter, an adaptive bisection that refines around the current best knee, a leave-one-out stability diagnostic, and a strict monotone polish.
3. A documented **methodology of rejected detours**. Several numerically attractive ideas (sandwich recalibration, λ-bisection, block-DP, sparse pairwise QUBO, top-K Hessian covering, surrogate-only knee, probe-only prediction) were proposed during design, some implemented; we report the conditions under which each is principled and the empirical reasons each was not adopted as the shipping path.

Headline empirical result. On Qwen3.6-27B, with identical source weights and identical export-time per-tensor tricks (HALO, GPTQ damp sweep, scale sweep, block-output match), PrismaSCOUT produces a 5.31 bpp artifact with held-out KL 0.0151. The previously published PrismaQuant v1 artifact at the same source was 5.50 bpp with held-out KL 0.0475 from the same calibration. The new artifact is 11% smaller and the held-out KL is 68% lower. Only the selection routine changed. A second selection-only upgrade — the *production-faithful* polish of Section 3.8, which evaluates candidate flips on the same quantized weights the exporter will ship rather than on RTN proxies and sweeps from a low-bpp floor under a small bits-budget creep — drives the same model further to 5.39 bpp / KL 0.0054, $\sim$2.8$\times$ lower KL than the shipped 5.31 artifact at +0.08 bpp.

The paper is organized as follows. Section 2 gives background and related work. Section 3 sets up the theoretical framework: the rate–distortion view, the additive surrogate failure mode, the cost hierarchy, the kneedle. Section 4 states the algorithm. Section 5 is a deliberate methodology section on the numerical detours we considered and rejected. Section 6 reports the engineering work that made the measurement loop tractable. Section 7 reports empirical results on Qwen3.6-4B and Qwen3.6-27B, including an honest accounting of the tool-eval-bench delta. Section 8 discusses limitations.

2 Background and Related Work

Per-tensor quantization. GPTQ [4] is the workhorse weight-only quantizer for transformers, posed as Hessian-weighted least squares over reconstruction error. AWQ [5] reweights salient channels. SmoothQuant [7] pushes activation quantization tractable by migrating activation magnitude into weights. AutoRound [6] sign-rounds via signSGD. HALO/HQQ-style rotations preprocess weight matrices to reduce outlier counts before integer quantization. These methods take a single matrix and a fixed format and produce a high-quality quantized version of that matrix; they do not decide *which* matrix gets which format.

Mixed-precision allocation. HAWQ-V3 [8] formulates mixed-precision quantization as an integer linear program over per-layer sensitivity (second-order Hessian information) and dyadic arithmetic constraints. CLADO [9] extends this with an explicit cross-layer dependency: pairwise quantization-error coupling is measured on a small data subset and the bit-allocation problem is solved as an integer quadratic program in seconds, rather than approximated additively across layers. Recent allocation work for LLMs includes CoopQ [10], which models layers as a cooperative game and uses a Shapley-based progressive allocator to capture inter-layer interaction, and AMQ [11], which applies AutoML-style search over per-layer weight-only bit-widths. Surveys of the broader mixed-precision space have appeared as the field has matured [12], and recent iterative allocators include IterQuant [13]. The geometric view of quantization error in [14] casts GPTQ as Babai's nearest plane algorithm and is orthogonal to allocation but informs how a fixed allocation should round.

Knee detection and operational rate–distortion. The kneedle algorithm of [16] provides a non-parametric estimator of the elbow of a tradeoff curve. The closely related operational rate–distortion (ORD) framing of [17] establishes that any discrete-choice rate–distortion problem admits a Lagrangian relaxation whose solutions trace the lower convex hull of the achievable points. Knees on the ORD frontier correspond to phase transitions where additional rate buys disproportionate fidelity. Our work imports the ORD vocabulary — and avoids the convex hull, since for mixed-precision LLM quantization the discrete frontier admits non-convex segments and a λ-sweep on the dual would skip them (Section 5.1).

Microscaling formats. The MXFP4/MXFP6/MXFP8 family is specified by the OCP microscaling working group [18]: 4/6/8-bit elements with shared FP8 (E8M0 or E5M2) block scales over groups of 32. NVFP4 is NVIDIA's narrower-group 4-bit format with 16-element FP8 group scales [19]. NVFP4 weight-only Linears cost $4 + 8/16 = 4.5$ bpp; MXFP4 weight-only Linears cost $4 + 8/32 = 4.25$ bpp. Modern Blackwell/Hopper GPUs offer first-class kernels for both. Mixed precision in this paper means an assignment $\mathcal{A} : \mathcal{L} \to \mathcal{F}$ where each Linear independently selects from {BF16, MXFP8, NVFP4, MXFP4, ...}, subject to fused-sibling constraints in attention (q, k, v) and MoE (gate_up, down).

Serving. The artifacts described here are exported to vLLM [20] via the `compressed-tensors` schema, with format coverage extending to NVFP4, MXFP8, BF16 passthrough, and (in MoE settings) MXFP4 experts. End-to-end inference uses Blackwell-native kernels.

2.1 The PrismaQuant v1 baseline and why it sets a hard bar

The artifact PrismaSCOUT replaces was not a weak baseline. The defining feature of PrismaQuant v1 is the *mixed-precision allocator itself*: each Linear in the model independently picks its serving format from {BF16, MXFP8, NVFP4, MXFP4, source-passthrough} under a global bits-per-parameter budget, rather than every Linear sharing one quantization scheme

as in single-format methods like GPTQ [4] or AWQ [5]. A tuned per-tensor toolkit runs *under* whatever format the allocator picks for a given Linear; the allocator decides where bits should be spent. v1 is worth describing in its own right both because PrismaSCOUT inherits it wholesale (only the selection routine changed in the head-to-head comparison of Table 1) and because understanding what v1 already captures clarifies which residual is left for joint-allocation methods to attack.

The mixed-precision allocator. For each Linear i in the model, v1 enumerates a candidate menu $\mathcal{F}_i \subseteq \{\text{BF16, MXFP8, NVFP4, MXFP4, source-passthrough}\}$ filtered by serving-time constraints (MXFP8 tile alignment for the target kernel, FlashInfer masks, source-dtype eligibility for passthrough), and assigns each candidate a per-Linear cost $u_i^{(1)}(f)$ from a HAWQ-style [8] sensitivity probe fed by a multi-chunk empirical-Fisher accumulator. A multi-choice knapsack DP

$$\min_{\mathcal{A}} \sum_i u_i^{(1)}(\mathcal{A}_i) \quad \text{s.t.} \quad \sum_i b_i(\mathcal{A}_i)\, n_i \le B \cdot \sum_i n_i$$

selects one format per Linear under the bits-per-parameter budget B, where $b_i(f)$ is the bit count for Linear i at format f and n_i is its parameter count. Fused-sibling coherence (q/k/v in attention, gate_up/down in MoE) is enforced natively by grouping siblings into refinement units that enumerate only joint sibling-coherent format combinations; joint-input-global-scale handling for fused attention is folded into the DP's transition costs. The output is a heterogeneous per-Linear assignment that may, for example, keep the highest-sensitivity attention output projection in BF16, the medium-sensitivity gate/up projections in MXFP8, the bulk dense MLPs in NVFP4, and (for native-FP8 source architectures) the down-projection verbatim from the source. This per-Linear heterogeneity is the operative quality lever: the shipped v1 Qwen-3.6 27B 5.5 bpp artifact, for example, mixes BF16, MXFP8, and NVFP4 Linears within a single model (the vLLM serve dispatches all three kernels at load time) — a distribution no single-format method can produce.

The per-tensor toolkit. For each Linear, after the allocator has picked its format, v1 composes:

- **HALO rotation preprocessing** [3] — a Hadamard-style orthogonal rotation applied to the weight matrix before integer quantization, which spreads outlier-channel mass over the rotated basis and reduces the per-element representation range. Particularly effective for 4-bit weight-only formats whose error is dominated by tail outliers.
- **Per-layer GPTQ** [4], the workhorse Hessian-weighted least-squares rounding. v1 ships an adaptive *damp probe* (commit `0cbaddb`) that tries multiple Hessian damping coefficients per Linear and picks the lowest measured output MSE; a *batched-NVFP4* fast path (commit `f57d960`) processes Linear groups together.
- **Scale sweep** (commit `7884536`) — a per-tensor scale-factor search that minimizes the post-quantization output MSE of each Linear, executed inside the export hot path.
- **Block-output matching** (commits `9ef0a05`, `a6c0cf6`, `b12f96b`) — after per-Linear quantization, a block-level optimization that re-tunes a subset of parameters so that the *block* output matches the BF16 reference, not just the individual Linear outputs.
- **Source passthrough** (commits `705fbcd`, `0bf4146`) — when the source checkpoint is already at a serving-compatible precision, the allocator may pick a verbatim copy of the source codes and scales rather than re-quantizing; the dequant view back to BF16 is lossless, so the per-Linear Δloss is exactly zero. This is important for native low-precision source architectures (MiniMax, DeepSeek-class) where re-quantization can only lose information.

- **Activation clipping and FP32 activation cache** (commit `a5d13db`) — wins the cost-step vs. activation quantization tradeoff for formats that benefit from clipped activations.

The whole pipeline (probe → allocator → per-tensor refinement → export) runs end-to-end on a UMA host with bounded memory.

Why this works as well as it does. The combination divides the quantization problem cleanly into two layers. The mixed-precision allocator solves the *global* question — which Linears tolerate 4-bit storage and which need 8 or 16 — and the per-tensor toolkit solves the *local* question — given a chosen format, what is the lowest-error rounding for this specific weight matrix. Each per-tensor component attacks one term of the per-Linear quadratic $\frac{1}{2}\langle H_i, (W_i - \hat{W}_i)^2\rangle$: HALO reduces $\|W_i - \hat{W}_i\|$ by spreading outliers; GPTQ minimizes $\langle H_i, (W_i - \hat{W}_i)^2\rangle$ under the chosen format; scale sweep tunes the post-rotation per-tensor scale to minimize the remaining error; block-output match closes the gap to the *block-level* reference, absorbing some of the inter-Linear coupling without an explicit cross-layer cost model. The HAWQ-style allocator then spends bits on the Linears whose local quadratic has the largest top eigenvalue. Each technique is narrow, principled, and composable inside its own layer; together they extract most of the per-Linear quantization headroom *conditional on* a fixed allocation.

The empirical evidence that this pipeline is strong is the community adoption: the public Qwen-3.6 27B, Qwen-3.5 122B-A10B, Qwen-3.6 35B-A3B, Mistral-Medium-3.5, MiniMax-M2.7, Gemma-4-31B, and DSv4-Flash PrismaQuant v1 artifacts have together accumulated over 60,000 downloads on Hugging Face in their first weeks of release, across a heterogeneous set of source architectures (dense, MoE, and native-FP8). v1 is a competitive shipping default in absolute terms.

The stacking-interference failure mode. Per-tensor techniques are conventionally treated as additive in single-format quantization, and on the per-Linear metrics each pass optimizes against this composition is empirically robust. In a mixed-precision setting it is not. Each pass operates on the activation distribution as modified by the previous pass, and the per-Linear cost models the passes consume are decoupled from the format choices the allocator eventually makes for upstream Linears. Pairs of individually validated wins can therefore interfere when stacked: an internal audit confirmed a recipe with all four wins enabled lost roughly half the gain of activation-clipping applied alone, with the do-no-harm gate reverting clip-improved Linears because its cached MSE comparison ran on the unclipped activation distribution while the upstream pass had optimized under the clipped one. AWQ [5] surfaces the same family from a different angle: it was defaulted off in v1 after empirical bakeoffs showed it *worsened* NVFP4 group quantization on most Linears, despite being a strict win on uniform-format 4-bit settings. The per-Linear "do no harm" gate (commit `ed9c60c`) is the direct band-aid: after GPTQ + scale-sweep, compare against pure RTN on the cached activations and revert that Linear if RTN beats the post-pass weight. Both observations are artifacts of the same structural property — per-tensor wins that modify the activation distribution do not propagate that modification forward into the cost models the downstream wins consume, so cumulative recipes are not, in general, the composition of their individually-validated parts.

What v1 leaves on the table. The systematic gap v1 cannot close from the per-layer view is the joint-allocation question: "which Linear should pay an extra bit?" Per-layer sensitivity correlates with end-KL impact but does not predict it perfectly, because end-KL depends on how a Linear's quantization error *propagates* downstream, and propagation is sensitive to the formats chosen for upstream Linears. v1's HAWQ-style allocator overspends on Linears with high local sensitivity but modest downstream impact, and underspends on Linears with low local sensitivity but errors that compound through later attention. The stacking-interference

failure mode of the previous paragraph is the mirror image of this gap on the per-tensor side: when upstream optimization decisions are not propagated into the cost model that downstream optimizations consume, the resulting cumulative recipe is not the composition of its individually-validated parts. PrismaSCOUT does not change a single per-Linear technique; it changes the allocator from *additive surrogate cost* to *validated end-KL on real assignments*, and it gates every shipping decision on a held-out KL measurement. The real-KL gate is also what catches the stacking-interference failure mode end-to-end — if a recipe stack quietly regresses on real KL, no DP rearrangement of the per-Linear costs hides that fact. The Qwen3.6-27B head-to-head in Table 1 is the empirical statement of this relationship: the per-Linear toolkit is held fixed, only the selection routine changes, and the resulting artifact is 11% smaller and 68% lower in held-out KL. The remaining headroom is at the joint level, which is where the rest of this paper goes.

3 Theoretical Framework

3.1 The shipping criterion

Let p_θ denote the full-precision teacher model with parameters θ and $p_{\hat{\theta}}$ a quantized variant with weights $\hat{\theta} = Q_{\mathcal{A}}(\theta)$ under format assignment $\mathcal{A}$. Given a calibration distribution $\mathcal{D}$ over input prefixes $x_{1:t}$, the operational quality of $\mathcal{A}$ is

$$D_{\mathrm{KL}}(\mathcal{A}) \;=\; \frac{1}{N\,T}\sum_{n=1}^{N}\sum_{t=1}^{T_n} D_{\mathrm{KL}}\Big[\, p_\theta(\,\cdot \mid x_{1:t}^{(n)}) \,\Big\|\, p_{\hat{\theta}}(\,\cdot \mid x_{1:t}^{(n)}) \,\Big], \tag{1}$$

that is, the per-token KL between teacher and quantized model averaged over $T = \sum_n T_n$ non-padding positions in N calibration prefixes.

KL is the natural objective for "does the quantized model behave like the teacher?" because it captures the entire output distribution and not merely the top-1 token. Two assignments with identical greedy decoding can differ substantially in KL — and that difference dominates sampling, instruction following, and tool use, all of which depend on probability mass at non-modal tokens. Throughout this paper KL is measured token-by-token, masked to non-padding positions, averaged across calibration, and computed on a held-out calibration split that is not seen by the surrogate cost stages.

3.2 The additive-surrogate failure mode

A natural objective for allocation is the additive sum of per-Linear costs

$$U_{\mathrm{add}}(\mathcal{A}) \;=\; \sum_{i\in\mathcal{L}} u_i(\mathcal{A}_i) \tag{2}$$

where $u_i(f)$ is the local cost of using format f at Linear i, holding all other Linears at some reference assignment $\mathcal{A}^{(\mathrm{ref})}$. The diagonal Hessian formulation $u_i(f) = \mathrm{Tr}(H_i)\cdot \mathrm{MSE}_i(f)$ used by HAWQ [8] is one example; output-MSE under fixed activation context is another. *For any choice of u_i that holds $\mathcal{A}_{-i}$ fixed, U_{add} is a first-order Taylor expansion of $D_{\mathrm{KL}}(\mathcal{A})$ centered at $\mathcal{A}^{(\mathrm{ref})}$.* A constrained minimization of U_{add} subject to a bit budget B via additive multi-choice knapsack DP greedily flips many Linears at once and routinely steps outside the trust region of the linear approximation. Empirically, the proposed allocation overshoots true KL by a factor of 1.3–1.5 on the configurations we measured; a 0.10 unit predicted decrease can produce a 0.08 unit measured *increase*.

This is the structural hazard the rest of the paper is built around. Figure 1 sketches the geometry.

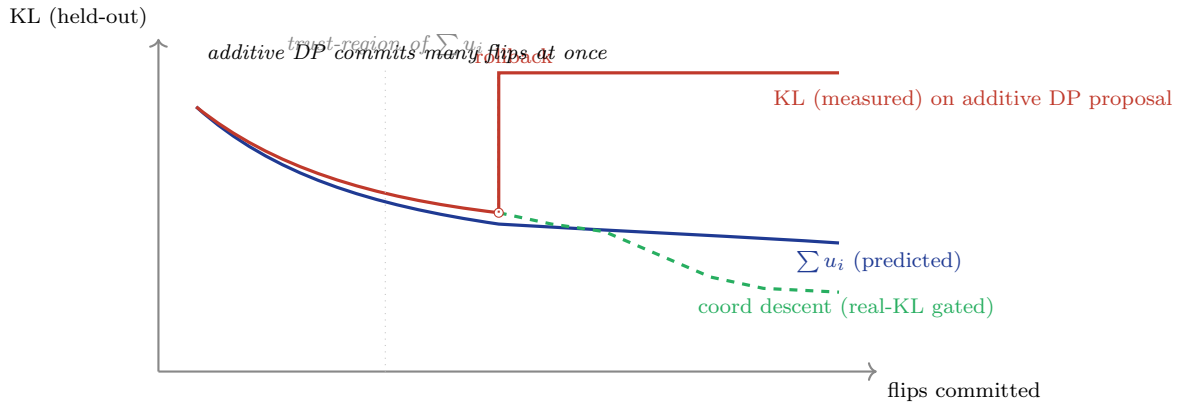

Figure 1: The non-additivity overshoot. The additive surrogate $U_{\text{add}} = \sum u_i$ predicts a monotone KL decrease as flips accumulate (blue). The real, measured held-out KL agrees with the surrogate inside the trust region of the local Taylor expansion, but diverges sharply when many flips are committed at once (red). The validated-frontier gate detects the regression and rolls back to the prior assignment; coord-descent then makes single-Linear flips, each gated on real KL (green dashed), and is monotone non-increasing by construction.

3.3 Surrogates generate, real measurement selects

The remedy adopted in PrismaSCOUT is to use U_{add} purely as a *candidate generator* and to select from generated candidates by real, end-to-end measurement of $D_{\text{KL}}(\mathcal{A})$ on a held-out calibration split. The objective inequality

$$U_{\text{add}}(\mathcal{A}) \le U_{\text{add}}(\mathcal{A}') \quad \not\Longrightarrow \quad D_{\text{KL}}(\mathcal{A}) \le D_{\text{KL}}(\mathcal{A}')$$

fails on average for many-flip moves. The substitute property

$$D_{\text{KL}}(\mathcal{A}) \le D_{\text{KL}}(\mathcal{A}') \quad \text{(measured on held-out)}$$

is observable for any pair we are willing to materialize and forward through the model. The cost of *making* that observation is what determines whether the surrogates-generate paradigm is practical at LLM scale, which is where the cost hierarchy enters.

3.4 The cost hierarchy

PrismaSCOUT runs three escalating cost models on the same problem (Figure 2).

L_1 probe ($\sim$ seconds per Linear).
A coarse per-matrix surrogate produced by a multi-chunk probe pass that accumulates an empirical Fisher trace $\text{Tr}(\hat{H}_i)$ alongside per-format output perturbation [3]. The allocator's preferred cost is $u_i^{(1)}(f) = \frac{1}{2}\langle \hat{H}_i^{\text{full}}, (W_i - Q_f(W_i))^2 \rangle$, the second-order Δloss proxy when a per-weight Hessian detail tensor is available; otherwise it falls back through measured local output MSE $\text{MSE}\big(W_i X^{(0)}, Q_f(W_i)\, X^{(0)}\big)$ and finally $\text{Tr}(\hat{H}_i) \cdot \text{MSE}_i^{\text{w}}(f)$, the Fisher-trace-weighted weight-MSE scalar (`measure_quant_cost.py`). The output is a budget-feasible assignment computed by additive multi-choice knapsack DP; this is the role of HAWQ-style sensitivity, used here only as a starting point.

L_2 perturbed-X fixed point ($\sim$ minutes).
A more careful per-Linear measurement that captures the activation distribution *under the current assignment* rather than under BF16. We install pre-quantization activation hooks

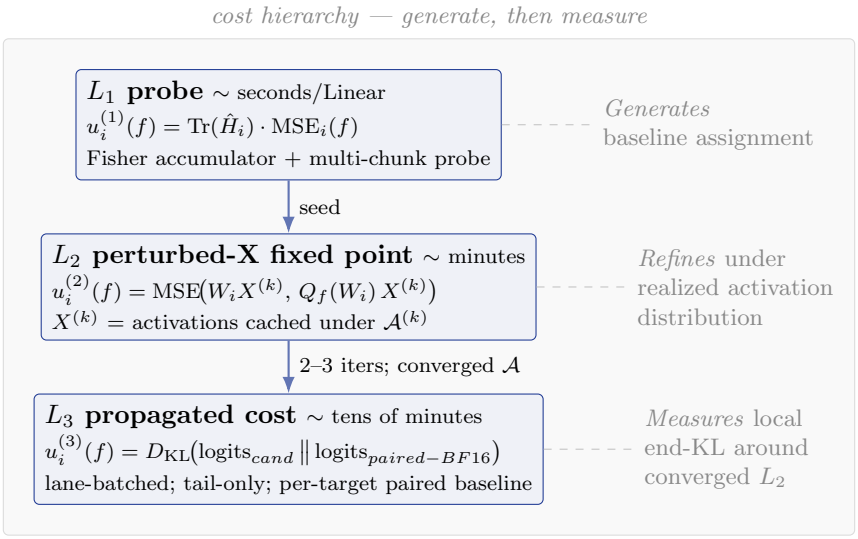

Figure 2: The three-level cost hierarchy. L_1 and L_2 are surrogate cost models used for candidate generation; L_3 is a real, end-to-end KL-divergence measurement against a target-specific paired baseline. The L_2 fixed point folds inter-layer coupling into the activation cache so that the per-Linear surrogate adapts to the realized assignment.

on the model under $\mathcal{A}^{(k)}$, forward calibration tokens, and cache the resulting activations $X^{(k)}$. We then measure

$$u_i^{(2)}(f) \;=\; \mathrm{MSE}\Big(W_i\, X^{(k)},\; Q_f(W_i)\, Q_f^{\mathrm{act}}(X^{(k)})\Big)\,,$$

where $Q_f^{\mathrm{act}}(\cdot)$ is the activation quantize-dequantize of the format under measurement (identity for weight-only formats; the matched activation quantizer for formats with native activation quantization), and the resulting scalar is fed through the same Δloss/Fisher-trace cost machinery as L_1. The DP solves under $u^{(2)}$, producing $\mathcal{A}^{(k+1)}$. The fixed-point iteration $\mathcal{A} \to X \to u^{(2)} \to \mathcal{A}$ typically converges in 2–3 iterations on the models we have measured. Convergence is declared when the weighted-Hamming flip ratio

$$h^{(k)} \;=\; \frac{\sum_{i:\, \mathcal{A}_i^{(k+1)} \neq \mathcal{A}_i^{(k)}} n_i}{\sum_i n_i}$$

(parameter-count weighted, with n_i the parameter count of Linear i) falls below `convergence-frac` (default 0.01). Inter-layer coupling enters through the activation cache: late-layer activations under a low-bit early-layer assignment differ from BF16 activations, and the per-Linear cost adapts.

L_3 propagated cost ($\sim$tens of minutes).
An end-KL measurement against a *target-specific BF16 paired baseline*. For each candidate (Linear ℓ, format f), we run two forwards: a baseline lane in which Linear ℓ is BF16 and every other Linear is at its L_2 assignment, and a candidate lane in which Linear ℓ uses format f. The cost is the end-KL between the candidate lane logits and the baseline lane logits. Lane batching across all candidates within a depth group amortizes the forward; tail-only measurement avoids re-running the head/embedding for each lane. The output is a per-(Linear, format) end-KL table $u_i^{(3)}(f)$ that the final DP consumes.

The three levels are wired in cascade: L_1 proposes, L_2 refines under the proposed activation distribution, L_3 measures the end-effect locally around the converged L_2 assignment. L_3

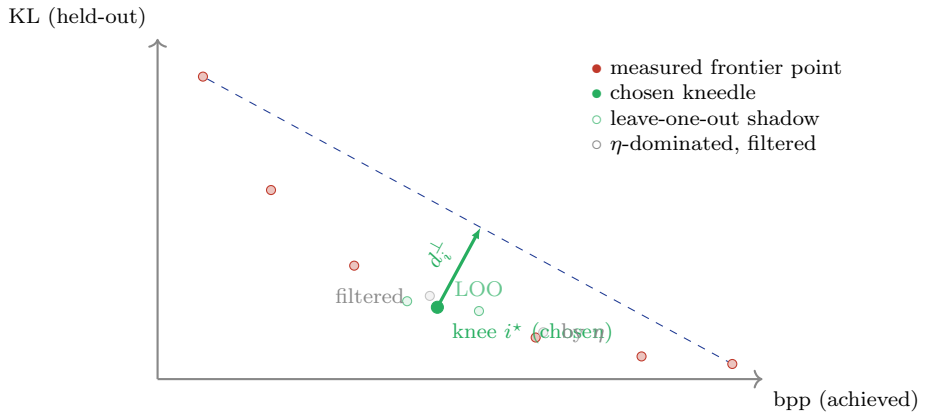

Figure 3: Validated-frontier kneedle. Each point on the frontier is a real, end-to-end held-out KL measurement of a materialized candidate assignment. The dominance filter widens by an explicit KL noise floor η. The chosen knee maximizes perpendicular distance to the endpoint-secant after min-max normalization (Equation 3). The leave-one-out diagnostic re-runs the kneedle on each subset of size $|F| - 1$; if the maximum bpp shift exceeds tolerance, the search emits an instability warning and records a guarded fallback knee.

semantics differ from L_1 and L_2 in a load-bearing way: L_3 *is* an end-KL measurement, just one whose paired baseline freezes the context outside the target Linear at the converged L_2 assignment rather than at BF16. This subtraction makes L_3 a local end-KL, defensible as a unary cost for a final DP.

3.5 The validated-frontier kneedle

The validated-frontier construction is sketched in Figure 3. After the cost hierarchy generates a set of candidate assignments $\{\mathcal{A}^{(j)}\}$ at multiple budgets, each candidate is materialized, forwarded against the held-out calibration split, and assigned a measured pair $(\mathrm{bpp}(\mathcal{A}^{(j)}), D_{\mathrm{KL}}(\mathcal{A}^{(j)}))$. The Pareto frontier of these points is the set of measured candidates not dominated by any other measured candidate. The dominance filter is widened by an explicit KL noise floor η:

Definition 1 (η-dominance)**.** $\mathcal{A}$ η-dominates $\mathcal{A}'$ if $\mathrm{bpp}(\mathcal{A}) \leq \mathrm{bpp}(\mathcal{A}')$ and $D_{\mathrm{KL}}(\mathcal{A}) + \eta < D_{\mathrm{KL}}(\mathcal{A}')$.

This avoids spurious points on the frontier created by KL measurement noise within η of a competing point. The kneedle is then computed on the η-non-dominated frontier as the point of maximum perpendicular distance to the secant joining the frontier endpoints, after standard min-max normalization of both axes. Concretely, with normalized $(\tilde{x}_i, \tilde{y}_i)$ on the frontier in increasing-bpp order, and $\tilde{y}$ replaced by $1 - \tilde{y}$ so the curve is increasing,

$$i^\star = \arg\max_i \frac{\left|(\tilde{y}_n - \tilde{y}_1)\tilde{x}_i - (\tilde{x}_n - \tilde{x}_1)\tilde{y}_i + \tilde{x}_n\tilde{y}_1 - \tilde{y}_n\tilde{x}_1\right|}{\sqrt{(\tilde{y}_n - \tilde{y}_1)^2 + (\tilde{x}_n - \tilde{x}_1)^2}}. \tag{3}$$

The kneedle is endpoint-fallback: if the maximum-perpendicular point is one of the two endpoints (a near-linear frontier), the central point is used instead and a warning is emitted. Adaptive bisection refines the frontier by evaluating the midpoint of the largest gap adjacent to the current best knee until either $|\mathrm{bpp}_R - \mathrm{bpp}_L| < \tau$ or the evaluation budget is reached.

3.6 Leave-one-out stability

A frontier with 5–7 points is small enough that the chosen knee can be brittle. We add a leave-one-out diagnostic: for each frontier point i, remove it and re-run the kneedle (Figure 4).

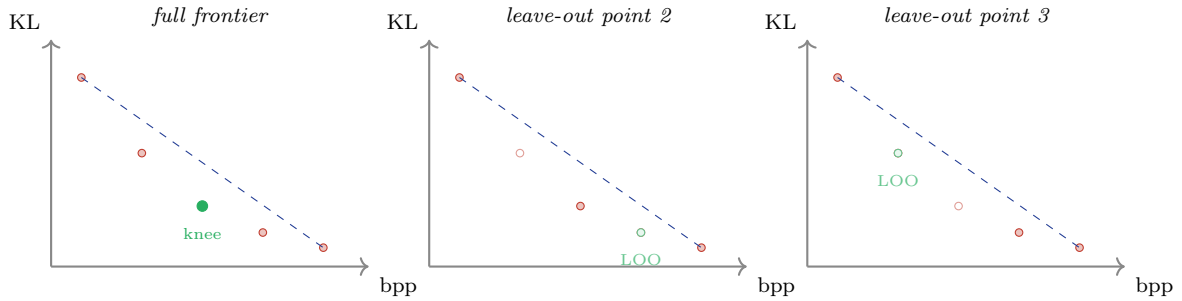

Figure 4: Leave-one-out kneedle stability. The diagnostic re-runs the kneedle algorithm on each subset of size $|F| - 1$ obtained by removing one frontier point. If the maximum bpp shift across removals exceeds tolerance, or the maximum KL shift exceeds the noise floor η, the search emits an instability warning and records both the original knee and a guarded fallback. In the synthetic example shown: removing a non-knee point (middle) leaves the knee location nearly unchanged; removing the knee point itself (right) shifts the LOO knee to a neighboring frontier point. A configuration that produces shifts of the latter type at every removal is an indication that the bracket is too wide or the frontier is under-sampled.

If the maximum bpp shift across removals exceeds tolerance, or the maximum KL shift exceeds η, the search emits an instability warning, and the artifact metadata records both the chosen knee and a guarded fallback. The default guard policy when the diagnostic flags instability is `best-kl`: replace the geometric kneedle with the lowest measured-KL frontier point. We do not hard-fail on instability; we record it. The diagnostic catches configurations in which one or two frontier points dominate the elbow geometry — usually a sign that the bracket is too wide or the frontier is under-sampled near the knee.

3.7 Coordinate descent as monotone polish

The polish trace is illustrated in Figure 5.

The polish does not produce a mathematical discovery; it provides a contractual guarantee that the shipped assignment cannot be worse than the chosen frontier point under the polish-time evaluator. We state the contract formally so that what it covers, and what it does not, is explicit.

Proposition 2 (Design guarantee, monotone non-regression of coord-descent polish)**.** *Fix a calibration prefix set C, a deterministic logits evaluator $\mathcal{E}$ (model class, kernel selection, RNG state, and quantization rounding mode all held constant), a fused-sibling-coherent move set $\mathcal{M}$, and a numerical-stability tolerance $\varepsilon \geq 0$. Write $K(\mathcal{A}) \equiv D_{\mathrm{KL}}(\mathcal{A}; C, \mathcal{E})$ for the KL measured against this fixed pair. Let $\mathcal{A}^{(0)}$ be the chosen knee assignment with $\kappa_0 = K(\mathcal{A}^{(0)})$. The coord-descent polish iterates single-refinement-unit flips $\mathcal{A}^{(t+1)} = \mathcal{A}^{(t)}[i \to f^\star]$ drawn from $\mathcal{M}$ and accepts only flips that satisfy $K(\mathcal{A}^{(t+1)}) < K(\mathcal{A}^{(t)}) - \varepsilon$. Then the output assignment $\mathcal{A}^{(\infty)}$ satisfies $K(\mathcal{A}^{(\infty)}) \leq \kappa_0$.*

Proof. Each accepted flip strictly decreases K on the fixed $(C, \mathcal{E})$ pair by more than ε; rejected flips leave the assignment and K unchanged. The sequence $\{K(\mathcal{A}^{(t)})\}$ is monotone non-increasing. □

The implementation uses $\varepsilon = 10^{-12}$ as a numerical-stability floor (strict less-than under floating-point roundoff; `coordinate_descent_polish` in `iterate_perturbed_allocation.py`). The proof is one line; the load-bearing content is the proposition's scope.

Remark 3. Proposition 2 guarantees non-regression on the calibration prefix set, the evaluator, and the move set fixed at polish time. It does not extend to (i) a different held-out calibration

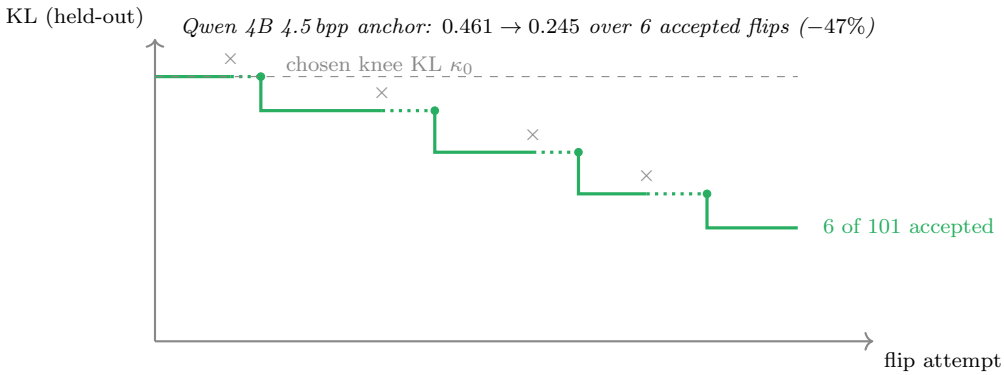

Figure 5: Coord-descent polish on the Qwen 4B 4.5 bpp anchor. Each attempt ranks the next single-Linear flip by L_3 unary cost; the flip is accepted only if real, held-out KL strictly decreases. Crossed marks are rejected attempts; filled circles are accepted flips. The KL trace is monotone non-increasing by construction. Six flips of 101 attempted moved KL from 0.461 (post-rolled-back L_3 polish) to 0.245.

set, (ii) a different evaluator (e.g. post-export inference engine, alternate kernel choice), or (iii) post-polish assignment mutation outside $\mathcal{M}$ (e.g. exporter-side fused-sibling rewrites, format coercions imposed by serving constraints). The shipped artifact is therefore re-validated end-to-end after export to bound the residual divergence between the polish-time evaluator and the production inference engine.

The operationally important point is that the polish does not need a cost model. It does not need to predict the change in KL; it measures it. The flip ranking comes from L_3 unary costs (cheap), the acceptance criterion is real KL on held-out (slow but rare). On Qwen3.6-4B at the 4.5 bpp anchor, coord descent committed 6 flips out of 101 attempts and moved KL from 0.371 (post-L_2) to 0.245, a 34% improvement. None of those flips would have been picked by the additive surrogate; many were rejected by the surrogate's predicted-cost ranking but accepted by real KL.

3.8 Production-faithful polish

Proposition 2 guarantees non-regression *at the polish-time evaluator*. Remark 3 flags that the shipped inference engine may diverge from the polish-time evaluator if they materialize quantized weights differently. The original coord-descent polish (`prismaquant.iterate_perturbed_allocation`) materializes NVFP4 / MXFP8 candidates by RTN-quantizing the BF16 source on-the-fly under the perturbed-X activation cache. The exporter, however, ships GPTQ reconstruction, scale-sweep, joint-NVFP4 sibling-coherent input global scales, and calibrated activation clip — per-Linear weights that can differ from RTN by several percent of relative Frobenius norm. The polish contract therefore guarantees non-regression on a proxy evaluator, not on the production engine. We close that gap with a second polish whose weight-materialization path is the exporter's.

Production weight cache. For each (Linear, format) pair we need to evaluate, `ProductionWeightCache` pre-renders the *exact* weight the exporter would ship: GPTQ, scale-sweep, joint sibling-coherent input global scales chosen across all members of a fused unit simultaneously, and a calibrated activation clip from the L_2 activation cache. Cache fill streams one Linear at a time and writes each rendered weight to disk (one `.pt` per (qname, fmt) pair) so a 27B-class model fits in the 121 GB UMA budget alongside the live model. A sidecar JSON records per-Linear max $|x|$ for resume.

Delta-quantize. Per-trial weight materialization on a 27B model is the dominant cost of polish: the naive path clones a slot the size of the candidate Linear's weight on every trial, which over a polish pass floods the UMA pool. `WeightSession` owns the model's quantizable parameters for the duration of polish: it materializes the starting assignment once on the live `model.params`, snapshots the BF16 source of each touched Linear lazily, and per-trial swaps a single unit's weight in place via `stage_unit`/`revert_unit_last` (or `commit_unit_last` on accept). The activation-quantization hook in `PerturbedActivationCache` is gated on `PRISMAQUANT_EXTERNAL_WEIGHT_MANAGEMENT` and skips its per-module clone-and-restore when set, since the WeightSession owns weight transitions. On Qwen3.6-27B this drops projected polish wall time from ∼80 minutes (and a peak ∼120 GB UMA pressure that OOMs the GB10) to 28 minutes at a steady ∼75 GB.

Polish-from-floor formulation. The production-faithful polish move set starts from a low-bpp *floor* (every quantizable Linear at the cheapest format that does not break a serving constraint, with `lm_head` pinned to BF16) rather than around the kneedle pick. The acceptance criterion is: real KL strictly decreases AND total bits stay within a small relative budget creep (default 5%). Polish accepts the flip with the largest real-KL drop among bits-feasible candidates and iterates until `max_passes` or a no-improvement pass. Polishing from a known floor is operationally simpler than polishing around an estimated knee and removes the surrogate-knee bias as a confound: the polish discovers, on production weights, exactly which subset of Linears to upgrade beyond the cheapest format, with no input from the additive surrogate beyond the floor itself.

Decision unit accounting. The move set in production-faithful polish is the set of Block-CLADO *decision units* from a probe payload (`prismaquant.block_clado.parse_payload`). A decision unit groups fused siblings (e.g. `q_proj`/`k_proj`/`v_proj`, or `gate_up_proj`/`down_proj`) that must share a format choice in vLLM's fused kernel. Each unit carries an explicit `member_qnames` list naming the Linear weights it controls. A defensive assertion in `coord_descent_polish` now raises if any unit has zero `member_qnames` present in the starting assignment: an earlier ad-hoc payload writer used the wrong key (`member_qnames` vs `members`) and `parse_payload` silently fell back to using the unit name as a sole pseudo-member, which made fused-sibling units invisible to polish and undercounted shipped bits by $\sim 2.4\times$ on 27B-scale runs (reported 5.07 vs actual 5.07 bpp floor; reported `final_bpp` 2.15 vs actual 5.07).

What changes structurally. The production-faithful polish picks a different concentration of high-precision weights than the upstream PrismaSCOUT kneedle. Section 7.4 reports the per-Linear disagreement: at 5.39 bpp the new polish keeps 12 Linears at BF16 and 5 at MXFP8, vs PrismaSCOUT's 137 BF16 and 1 MXFP8 at 5.31 bpp. The 12 Linears the polish kept high-precision are concentrated in `mlp.down_proj` at layers $\{0, 10, 16, 62\}$, `mlp.{gate, up}_proj` at $\{16, 19\}$, and a small set of embedding-adjacent and mid-network linear-attention projections. PrismaSCOUT's distributed BF16 budget — mostly small `linear_attn.in_proj_{a, b}` rank-projection matrices — is demoted entirely to NVFP4. Real-KL measurement on production-faithful weights judges the rank-projection matrices cheap to quantize and the `mlp.down_proj` bottleneck Linears expensive.

4 The Algorithm: PrismaSCOUT

4.1 Pipeline

The full pipeline is summarized in Figure 6 and Algorithm 1. For each anchor budget B:

1. **Probe (L_1).** Fisher-weighted MSE per (Linear, format). Solve additive DP at B to get $\mathcal{A}^{(0)}$.

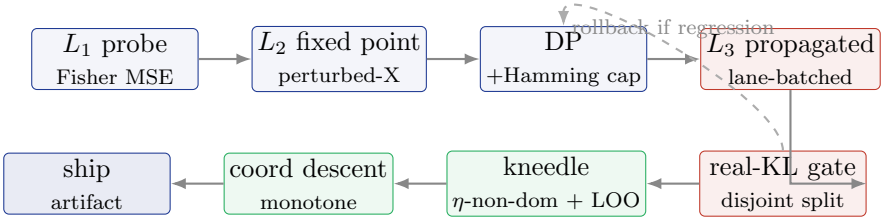

Figure 6: The PrismaSCOUT pipeline. Blue stages are surrogate candidate generation (L_1, L_2, additive DP). Red stages are real measurement (L_3 propagated end-KL, real-KL gate on disjoint validation split). Green stages are selection: validated-frontier kneedle and monotone coord-descent polish. Surrogate proposals are rolled back if the real-KL gate detects a regression.

2. **Perturbed-X fixed point (L_2).** Iterate $k = 0, \ldots, K$: install activation hooks under $\mathcal{A}^{(k)}$, cache calibration activations, measure $u_i^{(2)}(f)$ for all (Linear, format), solve weighted-Hamming-capped DP at B for $\mathcal{A}^{(k+1)}$. Stop on weighted-Hamming convergence ($K \leq 3$ in practice).
3. **Propagated cost (L_3).** Select a bounded neighborhood of Linears (uncertain choices, non-cheapest current picks, safety set); for each, measure $u_i^{(3)}(f)$ via paired BF16/candidate lanes, lane-batched, tail-only. Solve frozen DP over the neighborhood at B.
4. **Real-KL gate.** Materialize the proposed assignment, forward against the held-out calibration split, record measured KL. If KL regresses beyond $\rho \cdot \kappa_{\text{before}}$ (default $\rho = 0$), roll back to the prior assignment.

For the validated-frontier kneedle: repeat the above for a small set of anchor budgets, validate the resulting candidates on the held-out split, filter by η-dominance, and select via Equation (3) with adaptive bisection. Run leave-one-out (Section 3.6). Run coordinate-descent polish (Section 3.7). Record audit trail.

4.2 L_3 neighborhood selection

The L_3 measurement budget is bounded; we cannot afford to measure every (Linear, format) pair on a large model. The pipeline therefore selects a bounded *neighborhood* of candidate Linears around the converged L_2 assignment, prioritized as follows:

1. **Uncertain DP choices.** Linears whose L_2 second-best format is within `l3-uncertainty-rel-tol` (default 0.10) of the chosen format under $u^{(2)}$. These are the Linears the additive DP could plausibly have flipped under a small perturbation of the cost.
2. **Non-cheapest current picks.** Linears whose current L_2 choice is more expensive (in bits) than the cheapest measured format that does not break the bit budget elsewhere; these are the candidates for trading bits down with end-KL gating.
3. **High-cost safety set.** A small fraction (`l3-safety-fraction`, default 0.02) of the highest-predicted-cost Linears, regardless of uncertainty, to catch high-impact picks that were not flagged by the uncertainty filter.
4. **Fill to minimum coverage.** If the above sets jointly cover fewer than `l3-min-fraction` (default 0.05) of Linears, additional non-cheapest picks are added to reach that floor; the union is hard-capped at `l3-max-fraction` (default 0.30).

For each selected Linear, L_3 measures the current L_2 pick, one cheaper format, one more accurate format, and BF16 when available; the final DP runs only over candidates with measured L_3 costs and a measured current-format candidate.

Algorithm 1 PrismaSCOUT (validated-frontier knee mode)

Require: source weights θ, calibration splits C_{train}, C_{val}, format menu $\mathcal{F}_i$ per Linear, anchor budget set $\{B_j\}$, KL noise floor η, bisection tolerance τ

$u^{(1)} \leftarrow \textsc{Probe}(\theta, C_{\text{train}})$ ▷ L_1 Fisher-trace probe
for each anchor budget B_j **do**
  $\mathcal{A}^{(0)} \leftarrow \text{DP}(u^{(1)}, B_j)$
  for $k = 0, 1, \ldots$ **do** ▷ L_2 perturbed-X fixed point
    $X^{(k)} \leftarrow \textsc{ActCache}(\theta, \mathcal{A}^{(k)}, C_{\text{train}})$
    $u^{(2)} \leftarrow \textsc{MeasureLocalMSE}(\theta, X^{(k)}, \mathcal{F})$
    $\mathcal{A}^{(k+1)} \leftarrow \text{DP}_{\text{Hamming-cap}}(u^{(2)}, B_j, \mathcal{A}^{(k)})$
    if $h^{(k)} <$ `convergence-frac` **then break**
    end if
  end for
  $\mathcal{N} \leftarrow \textsc{L3Neighborhood}(\mathcal{A}^{(k)}, u^{(2)})$
  $u^{(3)} \leftarrow \textsc{MeasurePropagatedKL}(\theta, \mathcal{A}^{(k)}, \mathcal{N}, C_{\text{train}})$
  $\mathcal{A}_j \leftarrow \text{DP}_{\text{frozen}}(u^{(3)}, B_j, \mathcal{N}, \mathcal{A}^{(k)})$
  $\kappa_j \leftarrow D_{\text{KL}}(\mathcal{A}_j; C_{\text{val}})$ ▷ real-KL gate
  if $\kappa_j > D_{\text{KL}}(\mathcal{A}^{(k)}) + \rho\, D_{\text{KL}}(\mathcal{A}^{(k)})$ **then** $\mathcal{A}_j \leftarrow \mathcal{A}^{(k)}$ ▷ rollback
  end if
end for
$\mathcal{F} \leftarrow \textsc{Pareto}_\eta\{(\text{bpp}(\mathcal{A}_j), \kappa_j)\}$ ▷ η-non-dominated
$\mathcal{A}^\star \leftarrow \textsc{Kneedle}(\mathcal{F}; \tau)$ ▷ Equation (3)
$\mathcal{A}^\star \leftarrow \textsc{LeaveOneOutGuard}(\mathcal{F}, \mathcal{A}^\star)$
$\mathcal{A}^\star \leftarrow \textsc{CoordDescent}(\mathcal{A}^\star, C_{\text{val}})$ ▷ Proposition 2
return $\mathcal{A}^\star$

4.3 The DP and fused-sibling constraints

The allocator solves an additive multi-choice knapsack DP whose *units* are not individual Linears but *refinement units* that group fused siblings. Attention q, k, v projections share a per-tensor global scale in vLLM's fused kernel [20]; in MoE, gate_up and down packed Linears must share schemes within an expert column. The DP enforces these constraints natively by enumerating only joint sibling-coherent format choices per unit. Fused-sibling aggregation uses sum (joint-cost) for activation-coupled siblings and max for isolated-sensitivity siblings; we report the rationale and an empirical comparison in Appendix A.

4.4 Calibration disjointness

The split discipline is summarized in Figure 7. PrismaSCOUT maintains two disjoint calibration splits:

- **Train split** (`l3_calib_split=train`, default). Used for L_2 activation caching and L_3 paired-baseline measurement.
- **Validation split** (`kl_calib_split=validation`, default). Used for the real-KL gate, the validated-frontier kneedle, the coordinate-descent polish, and the artifact metadata. Never seen by any cost-generation stage.

The renaming and split discipline was added in commit `2e670b3` ("Implement PR-Polish-1: Rename validation_kl and secure surrogate knee") after an audit identified that the original `validation_kl` field was, in fact, an in-sample number on the same calibration split that drove L_2/L_3 cost generation. Until that fix landed, `validation_kl` could be reasonably misread as a held-out metric; it was not.

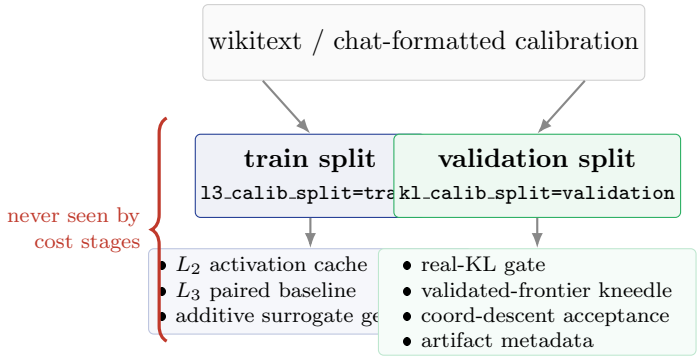

Figure 7: Calibration split discipline. The train split feeds all surrogate cost-generation stages (L_2 activation cache, L_3 paired baseline, additive DP). The validation split feeds every selection decision (real-KL gate, kneedle, coord descent, artifact metadata) and is never seen by any cost stage. Until commit `2e670b3` (PR-Polish-1), the field labelled `validation_kl` could be reasonably misread as held-out; that field has been renamed throughout to require disjoint splits in the shipping path.

5 Methods Considered and Rejected

The shipping algorithm above is the result of a deliberate triangulation between several principled alternatives. We describe each, the math behind it, and the empirical or structural reason it was not adopted as the default. The intent is not exhaustiveness for its own sake — it is to make explicit that simpler, mathematically attractive answers were considered first. Figure 10 catalogs the full set, classified as shipped, experimental, or rejected.

5.1 Lagrangian λ-bisection on the dual gradient

The proposal. The constrained problem $\min_{\mathcal{A}} U(\mathcal{A})$ s.t. $B(\mathcal{A}) \le B$ admits the Lagrangian relaxation $L(\mathcal{A}, \lambda) = U(\mathcal{A}) + \lambda(B(\mathcal{A}) - B)$. Sweeping λ traces the lower convex envelope of the achievable (bpp, U) frontier. For each λ, the per-Linear minimizer decouples: $\mathcal{A}_i^\star(\lambda) = \arg\min_f \; u_i(f) + \lambda\, b_i(f)$ where $b_i(f)$ is the bit count for Linear i at format f. This costs CPU seconds. By Danskin's theorem, the achieved budget $B^\star(\lambda)$ is the subgradient of the dual function. The kneedle is the region of maximum $|dB^\star/d\lambda|$ on log-λ. A few-step bisection on λ produces the kneedle without sweeping the full grid.

Theoretical optimality. For a strictly convex frontier, the proposal is optimal: 5–7 λ evaluations isolate the kneedle to within 0.04 bpp.

Why we did not adopt it as the default. Two reasons.

First, the discrete frontier is not strictly convex. Sub-bit combinatorial packing produces concave pockets where a slightly suboptimal assignment lies *above* the convex hull. No strictly positive λ selects those points as global minimizers of L; sweeping λ leaps over them. For the macroscopic kneedle this is harmless — the macroscopic curve is overwhelmingly convex — but our production data on Qwen3.6-27B exhibits a non-trivial gap between the surrogate-only knee at 5.857 bpp / measured KL 0.056 and the validated-frontier knee at 5.31 bpp / measured KL 0.015 (Figure 8). The validated-frontier kneedle and the discrete archive-DP candidate set both find the 5.31 point; a pure λ-sweep on the surrogate does not.

Second, the surrogate U_{add} used inside the Lagrangian is the same additive quantity whose joint behavior overshoots true KL by 30–50%. Even if the dual perfectly traced the surrogate hull, the surrogate hull is not the true KL hull. We therefore retained a λ-*archive search* as an experimental candidate generator (commit `86f2c14`, "Add lambda archive knee search"; commit

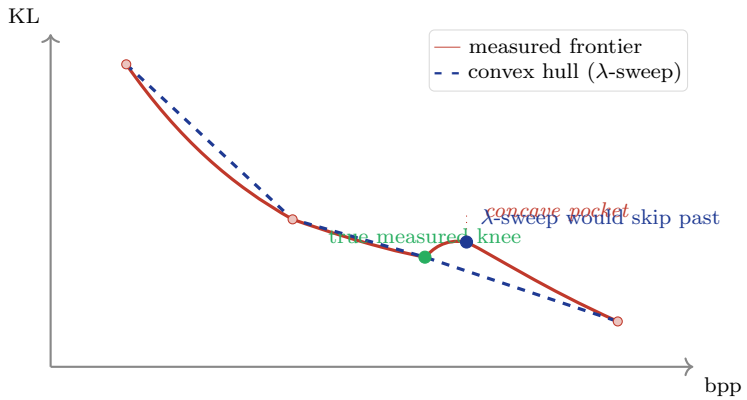

Figure 8: Why a Lagrangian λ-sweep is not the shipping selector. The achievable mixed-precision frontier (red) admits non-convex "pockets" — combinatorially-suboptimal assignments that lie *above* the lower convex hull. A λ-sweep traces the dashed hull and never selects pocket points; for the macroscopic kneedle this is harmless, but on the Qwen3.6-27B production data the gap between the hull-knee at 5.857 bpp and the measured-frontier knee at 5.31 bpp is operationally meaningful. We retain the λ-archive search as a candidate generator and gate selection on real KL.

`444bc28`, "Add one-pass Pareto archive DP"), with a beam variant retaining alternate states per bit bin. But the final selector is real-KL on the validated frontier; the λ-archive feeds candidates into that selector, not the other way around.

5.2 Sandwich proximal recalibration

The proposal. The non-additivity bug is a Taylor-trust-region bug: $u_i^{(2)}$ is measured holding $\mathcal{A}_{-i}$ at $\mathcal{A}^{(\text{ref})}$, the DP then proposes a multi-flip $\mathcal{A}^{(\text{prop})}$ far outside that radius. The remedy proposed in the design deliberation [3, `scratch/deliberation/02_gemini.md`] is a *sandwich*: propose $\mathcal{A}^{(\text{prop})}$ from the additive DP, re-measure $u_i^{(2)}$ holding $\mathcal{A}_{-i}$ at $\mathcal{A}^{(\text{prop})}$, re-solve the DP. Under a Mirror-Descent analysis with contraction factor ρ equal to the ratio of off-diagonal to block-diagonal Hessian energy — which the deliberation estimated at $\rho \in [0.3, 0.5]$ from the observed $\text{KL}_{\text{prop}}/\text{KL}_{\text{DP}} \in [1.3, 1.5]$ overshoot ratio — 2–3 sandwich passes are predicted to reach within 5% of the true KL minimum. We did not run the sandwich loop end-to-end, so this estimate is analytic, not empirical.

Why not adopted as default. Two practical reasons.

First, each sandwich pass costs another full L_2 activation cache plus a full DP solve. On Qwen3.6-27B, each pass is ∼22 minutes wall, so 3 passes is over an hour per anchor. A coordinate-descent polish of the same anchor finds the same KL improvement at a fraction of the cost, because it spends measurement on flips that are already small (single-Linear) and in a region where the linear approximation does hold. Coord descent is also trivially monotone on real KL by construction, while sandwich is *empirically* contractive.

Second, the operational re-centering target is the *actual* chosen assignment, not a sequence of L_2 DP outputs. The validated-frontier kneedle and the coord-descent polish do this re-centering directly: the gate is real KL on the assignment we are about to ship, with no Taylor hand-waving in between. Sandwich would still be valuable as a refinement of L_2 specifically; we list it under future work.

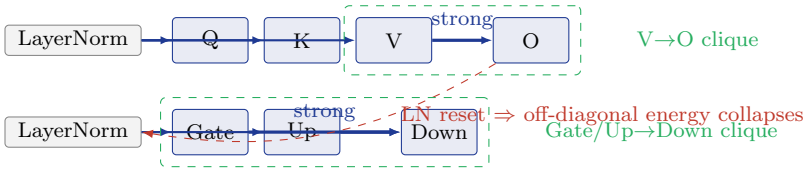

Figure 9: Architectural cliques in transformer blocks. The dominant non-additive interactions in mixed-precision quantization are confined to the V→O attention output projection and the Gate/Up→Down MLP output projection. LayerNorms reset error magnitude between blocks, so cross-block off-diagonal Hessian energy is small. A block-DP over these disjoint cliques (Section 5) would in principle capture the structural interaction without an NP-hard QUBO. We do not adopt it as a default because the additive DP's fused-sibling refinement units already absorb the within-clique constraint via joint sibling-coherent format choices.

5.3 Block-DP over architectural cliques

The proposal. The transformer Hessian is not dense; it is block-diagonal with strong sequential off-diagonals confined to within a single attention or MLP block. The structurally non-zero pairwise interactions are essentially:

$$\mathrm{V}_{\mathrm{proj}} \to \mathrm{O}_{\mathrm{proj}}, \qquad \mathrm{Gate}_{\mathrm{proj}}/\mathrm{Up}_{\mathrm{proj}} \to \mathrm{Down}_{\mathrm{proj}}.$$

A block-DP over these cliques (size 2–3, disjoint after LayerNorm resets, illustrated in Figure 9) captures the dominant interaction energy without an NP-hard QUBO, in $K = 2 \cdot \mathrm{num\_layers}$ measurements per pass.

Why not adopted as default. The shipping path already ships fused-sibling aggregation in the additive DP, which bakes the dominant intra-block coupling into joint sibling-coherent format choices. The Gate/Up → Down correlation is already treated jointly inside the refinement unit. Block-DP would extend this to V → O, which is real, but not large enough on the configurations we measured to justify a measurement pass beyond the existing sibling aggregation. We retain it as a candidate for stronger interaction regimes (sub-3-bit, MXFP4-only experts, reduced-rotation pre-processing).

5.4 Sparse pairwise QUBO / SMRF

The proposal. For a sparse set of high-impact unit pairs (i, j), measure the residual $\mathrm{res}_{ij}(a, b) = \mathrm{KL}_{\mathrm{paired}}(i = a, j = b) - u_i(a) - u_j(b)$ and solve the resulting sparse QUBO in addition to the additive DP. This is implemented in `interaction_refine.py`.

Why default-off. On the production Qwen3.6-27B run, the pairwise stage covered 8 high-impact units at the cost of ∼250 paired forwards (∼45 minutes) and rolled back when validation regressed. Coverage of 8 units out of ∼500 Linears is what one of our reviewers (Gemini 3.1) called *homeopathic*: too local to fix global non-additivity and expensive enough to dominate the budget when activated. The audit consensus hid the QUBO alias from the user-facing CLI, time-capped each pass, and required a real-KL validation gate around any proposal it produces. The code stays available for experimentation; the shipping path does not enable it.

5.5 Top-K Hessian-eigenvalue covering

The proposal. Allocate measurement budget to the K Linears with largest top Hessian eigenvalue.

Why rejected. The eigenvalue ranks individual matrices by local sensitivity. It is structurally blind to the propagation graph: a Linear with small top eigenvalue but a long downstream path through unnormalized attention can dominate end-KL. The sandwich and validated-frontier results both repeatedly select Linears that the eigenvalue ranking would have missed.

5.6 Surrogate-only knee selection

The proposal. Sweep target bpp on a single cached L_3 cost table; let $U_{\text{add}} = \sum_i u_i^{(3)}(\mathcal{A}_i)$ stand in for KL; pick the kneedle on the surrogate frontier.

Why rejected. On the Qwen3.6-27B production data (Figure 13), the surrogate kneedle picks 5.857 bpp / KL 0.056. The validated kneedle on the same candidate set picks 5.31 bpp / KL 0.015. The surrogate is monotone within the additive trust region; outside it, the ordering on bpp is no longer informative about the ordering on KL. We retain the surrogate sweep as an extremely cheap candidate generator (CPU seconds for hundreds of points) and pair it with an obligatory `--knee-surrogate-validate` gate that re-measures the picked points on real KL. After PR-Polish-1, the surrogate-only path is hidden from the user-facing CLI; outputs are relabeled `candidate_surrogate_*` so a reader cannot mistake the surrogate kneedle for the shipping artifact.

5.7 Probe-only knee prediction

The proposal. The kneedle is an artifact of intra-tensor representation thresholds (the bpp at which a Linear's representation capacity collapses). The L_1 probe captures those thresholds well. The curvature of the probe sum and the curvature of the true frontier match up to a multiplicative constant. Predict the knee bpp from L_1 alone.

Why used as a prior, not a selector. On Qwen-4B-class scales the expected error in the probe-predicted knee is empirically ± 0.12–0.18 bpp [3]. The density of mixed-precision states near the true knee is often ≤ 0.05 bpp. A ± 0.15 bpp error therefore guarantees we miss the exact discrete kneedle. The probe is the optimal prior for *bracket* establishment (e.g., search in $[3.8, 4.1]$ bpp), but the final discrete selection requires a real-KL evaluation of 3–5 λ- or DP-generated candidates within that bracket.

5.8 Pareto archive DP and beam variants

The proposal. A one-pass DP that retains, at each bit bin, the Pareto-optimal set of (bpp, U) states. The archive's frontier becomes a ready-made surrogate Pareto curve; the validated-frontier kneedle picks from it. The beam variant (`--knee-archive-beam-per-bin`) retains k alternate states per bin. The refinement stage adds neighbors of the current best measured KL.

Status. Implemented and flag-gated as candidate generation (commit `444bc28` for one-pass DP, beam variant in commit `53fe0db`). The archive feeds candidates into the validated-frontier selector when enabled via `--knee-archive-*` flags; it does not replace the validated-frontier kneedle as the final selector. The production Qwen3.6-27B archive run (`27b-knee-archive-beam4-refine-noseed`) is the canonical example: its no-seed selection was 5.31 bpp / measured KL 0.0147, about 9.9% relative KL worse than a separately-supplied best assignment at 5.27 bpp / KL 0.0134. We treat archive DP as a high-coverage candidate generator, audited by the same real-KL gate as the rest of the pipeline.

5.9 Top-down (ceiling-start) production-faithful polish

The proposal. Mirror the bottom-up production-faithful polish (Section 3.8) by starting at the all-BF16 ceiling and accepting flips that strictly decrease total bits while keeping real KL within a small budget. Two trajectories from opposite endpoints ought to sandwich the achievable Pareto interior, and their union ought to give much denser frontier coverage than either alone — a symmetric two-trajectory sandwich.

Why default-off. On Qwen3.6-27B, the bottom-up forward pass committed 12 accepted moves in 1706 s and reached the 5% bits-budget creep. The top-down pass with the symmetric `max_passes`=12 cap committed 12 accepted moves in 13 s — two orders of magnitude faster wall-time per accept — and consumed only 0.05% of its KL budget (0.0023 of an allowed 0.05). The two trajectories did not overlap: forward stayed near 5.39 bpp, top-down stayed near 15.62 bpp, and the entire interior was empty. The cause is the move set's geometry: starting from all-BF16, every candidate flip is a $\sim$ 12-bit reduction (BF16 $\rightarrow$ NVFP4) at near-zero KL cost, so the greedy chooses the unit with the smallest KL increase among those, finds many nearly-identical such picks at the start, and uses the entire `max_passes` budget on cheap flips with no incentive to extend toward the bpp range where the forward sweep operates. Increasing `max_passes` to 100+ would in principle close the gap, but the top-down direction also has no convergence guarantee *toward a knee*; it converges toward a budget-feasible all-NVFP4 floor that the bottom-up sweep already produces directly. We retired the top-down direction; the shipping production-faithful polish runs only the bottom-up sweep from the floor.

What we kept from the experiment. The plumbing (`--direction top_down`, `--kl-budget`) stays in `polish_from_assignment` for symmetric experimentation, gated off by default. The defensive `member_qnames`-coverage assertion in `coord_descent_polish` that this round added is unconditional — both directions would have silently produced wrong bits accounting under the original payload-key bug.

5.10 The UMA-clone misadventure

The proposal. On a 121 GB UMA host, polish on a 27B model runs out of memory if every trial clones the candidate Linear's weight in GPU-resident memory. A natural-looking remedy is to clone the slot to *CPU* between trials and copy back on demand: CPU memory and GPU memory are different pools, so the trial-time peak should drop.

Why structurally wrong. The GB10 is a unified-memory device: the CPU and GPU share the same physical pool. "Move to CPU" relabels a tensor as host-resident in the PyTorch view but does not change which DRAM page backs it; the trial-time peak is identical. The right fix is not to clone at all per trial, which is the `WeightSession` delta-quantize path (Section 3.8). We mention the non-fix because it is a recurring class of mistake on UMA architectures: a memory-management heuristic that is correct on a discrete-GPU host can be a no-op on a UMA host, and the eviction path of the `ProductionWeightCache`'s LRU was designed against a discrete-GPU mental model before being re-derived under the UMA assumption. The production cache now disk-streams (one `.pt` per (qname, fmt) pair) rather than relying on host/device pool separation, with an LRU bound on resident-tensor bytes.

6 Engineering Considerations

The theoretical framework requires real-KL measurement to dominate selection. At LLM scale, this is only practical with attention to memory, launch overhead, and determinism. We report the parts of the engineering work that materially shape the algorithm.

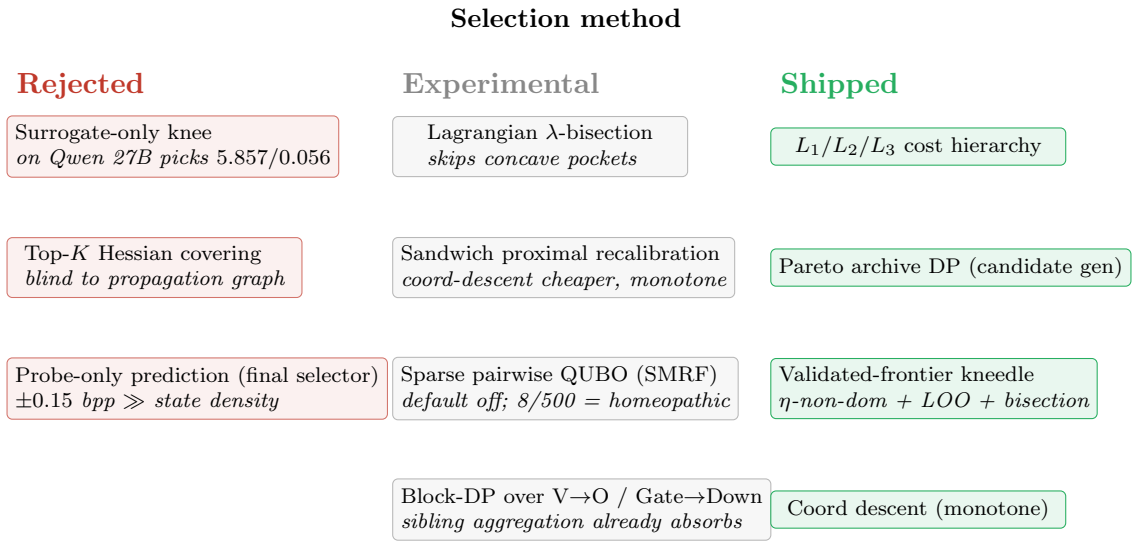

Figure 10: Methods considered during design, classified by status. *Shipped* (green) is the production path. *Experimental* (gray) is implemented in the codebase but disabled or hidden from the user-facing CLI; remains available for stronger interaction regimes. *Rejected* (red) was proposed and analyzed but did not survive empirical or structural analysis. Each method is principled in its regime; the column it lands in reflects whether that regime matches modern transformer mixed-precision allocation under a real-KL gate.

6.1 The 128 GB UMA constraint

The development host is an NVIDIA GB10 (Blackwell) with 128 GB unified memory shared between GPU and host. Quantization workloads at Qwen-27B scale have a non-trivial peak memory schedule (Figure 11): model weights ∼54 GB BF16, L_2 activation cache up to 24 GB across iterations, frozen-weight cache ∼4 GB, lane-batched activations ∼3−5 GB, CUDA graph pools ∼10−20 GB if naively stacked, Triton kernel JIT transients ∼1−2 GB, and Python plus framework baseline ∼3 GB. Single-component runs fit cleanly. All-on stacks reach 99–120 GB and OOM-kill, sometimes silently before the in-process budget enforcer can intervene.

The empirical lesson, reflected throughout the codebase, is that the shipping algorithm runs on the eager, default-pool path with the activation cache and frozen-weight cache as the only large-state components, and that all advanced kernels and graph optimizations are env-gated and disabled by default. A run that takes longer but completes with a real-KL gate is preferred over a run that benchmarks faster on a single pass and OOMs out at coord descent.

6.2 Lane-batched L_3 measurement

The L_3 measurement of one (Linear, format) candidate forwards a calibration batch through the model with one Linear in BF16 and one Linear at the candidate format. Naively, each candidate is a separate forward. Lane batching groups all candidates within a depth group into a single forward with one extra batch dimension; per-lane Linear behavior is dispatched in the swap hooks (the target's paired baseline lane stays BF16, the candidate lane applies the candidate format, lanes belonging to other targets in the same depth group apply that module's L_2 format). This reduced launch overhead by 30−60× versus the sequential implementation, at the cost of activation memory linear in the number of lanes; the `--l3-max-lanes-per-batch` flag bounds memory.

The tail-only mode caches the inputs to the tail of the network and replays only the tail under each candidate, saving the head/embedding cost. `PRISMAQUANT_L3_MIN_HOST_MEM_GB` adds memory backpressure for UMA hosts: the loop checks `/proc/meminfo` `MemAvailable` between

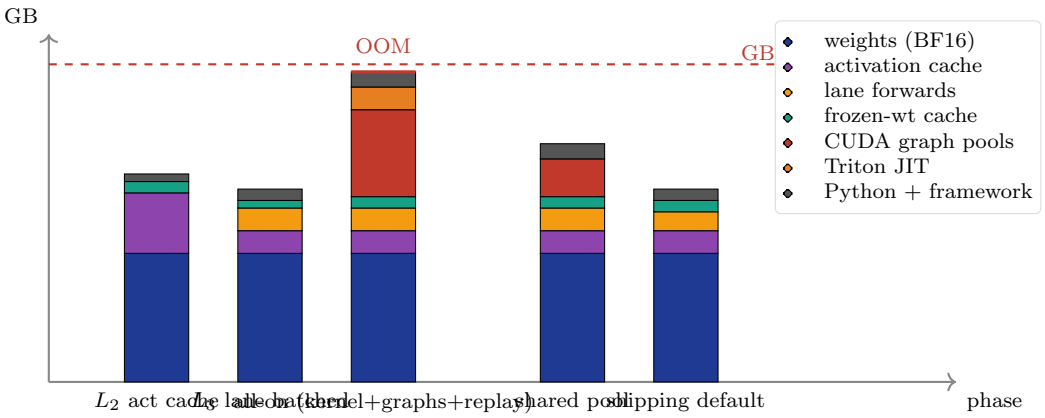

Figure 11: Memory landscape on the GB10 development host (Qwen3.6-27B configuration; bar heights illustrative, drawn to relative scale). Single-component runs fit cleanly. The naive "all-on" configuration (per-path CUDA graph pools + Triton-fused NVFP4 kernel + replay cache) stacks past the 128 GB UMA cap and OOM-kills. The shared-pool configuration consolidates four graph pools into one workspace and recovers ∼10 GB. The shipping default eager-pool path with bounded LRU caches is the rightmost bar; it is what produces the published artifacts.

paired-override chunks and reduces the next chunk's lane count before the host reaches OOM pressure.

6.3 CUDA graphs and the shared-pool retreat

CUDA graph capture eliminates per-launch overhead for the fixed-shape forward paths used in L_2, L_3, KL evaluation, and coordinate-descent flip evaluation. Naive capture allocates a separate graph memory pool per path, so 4–5 pools stack to ∼10−20 GB. We attempted a shared-pool implementation (commit `3f320c7`) that captures all paths into a single workspace with a tracked allocator. The shared-pool implementation exposed a smoke-test NaN under one configuration (commit `d830a01`, "graphs: default shared pool OFF after smoke NaN risk"); a second pass re-enabled shared-pool by default after a determinism flag and a safety-override no-clone path were added (commits `2ba832f` and `d9d67a9`). Through this, every path remains env-gated: `PRISMAQUANT_L3_CUDA_GRAPHS`, `PRISMAQUANT_COORD_LANE_CUDA_GRAPHS`, `PRISMAQUANT_KL_CUDA_GRAPHS` accept `auto`/unset, `1`, or `0`, and auto avoids graphing one-shot flip batches because capture warmup dominates on small workloads.

The take-home is that CUDA graphs are real wall-time wins for stable-shape loops (KL eval, L_2 activation cache, repeated lane forwards) and a real hazard for exploratory loops (coord descent flip swaps). The shipping default is auto: graph the stable paths, eager the rest.

6.4 Determinism and reproducibility

The audit trail recorded with each artifact includes: every measured (bpp, KL) point on the validated frontier, the dominance filter parameters (η and the bisection tolerance), the chosen knee, the leave-one-out shadow knees, the coord-descent flip log, the L_2 iteration trace, the L_3 neighborhood selection rationale, and the final assignment hash. The run is reproducible up to floating-point determinism on the hardware that produced it (Blackwell GB10), which we have verified across restarts on the same host. Artifacts are gated by an artifact registry (`prismaquant.artifact_registry`, commit `b3d8445`) that refuses to publish without a passed validation entry.

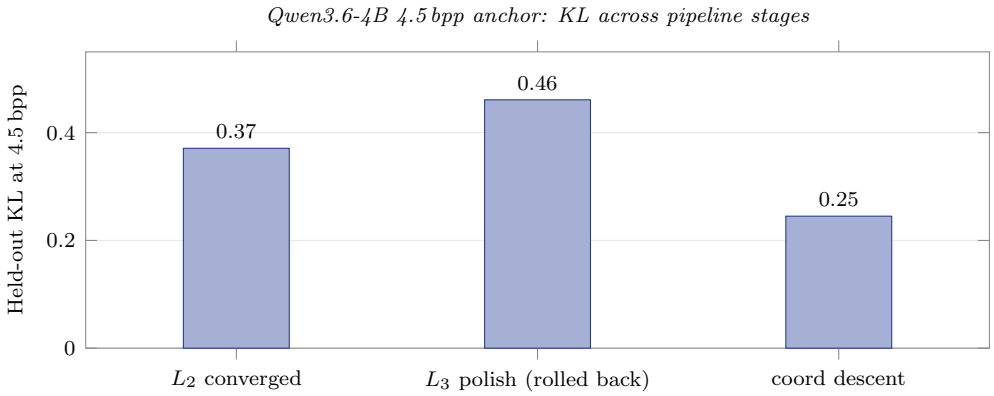

Figure 12: Pipeline progression at the Qwen3.6-4B 4.5 bpp anchor. L_2 converges to KL 0.371. The L_3 polish DP applies the additive-DP non-additivity bug at scale and *regresses* to KL 0.461 (+24%); the real-KL gate detects the regression and rolls the polish output back to the prior assignment. Coord descent then commits 6 single-Linear flips of 101 attempted, each gated on real held-out KL, ending at KL 0.245 (−34% versus L_2). The pattern is reproducible across every 4B anchor we have measured and is the empirical motivation for separating candidate generation from candidate selection.

7 Empirical Results

7.1 Qwen3.6-4B at the 4.5 bpp anchor

The 4.5 bpp anchor on Qwen3.6-4B is the canonical microbenchmark for the non-additivity story. Figure 12 reports KL at three stages of the pipeline.

The L_3 polish stage *regresses* relative to L_2 (0.371 → 0.461, +24%), consistent with the additive overshoot mechanism described in Section 3; the regression is detected by the real-KL gate and the polish output is rolled back. The coordinate-descent stage recovers and surpasses both: 0.461 → 0.245, with 6 flips committed out of 101 attempts. The shipped 4B artifact at this anchor is the coord-descent output. The same pattern reproduces at every Qwen-4B anchor we have measured.

7.2 Qwen3.6-27B production

The Qwen3.6-27B PrismaSCOUT artifact (DOI `10.57967/hf/8656`, [1]) is produced from the same source weights and the same per-tensor toolkit (HALO, GPTQ damp sweep, scale sweep, block-output match) as the previously shipped PrismaQuant v1 5.5 bpp artifact [2]. Only the selection routine changed. Both artifacts are the empirical operating point each selection regime deployed: 5.5 bpp was where v1's sensitivity-driven allocation, run under the same per-tensor toolkit, produced the model that v1 deemed worth shipping; 5.31 bpp is PrismaSCOUT's validated-frontier kneedle (Section 3.5). The head-to-head therefore measures what happens when the selection regime changes around a fixed candidate-generation pipeline, not what happens when the budget is held fixed and one asks which regime quantizes better at that bpp.

The artifact is 11% smaller and the held-out KL is ∼ 3.2× lower. The validated-frontier picture (Figure 13) is the algorithmic explanation: the surrogate-only kneedle on the same candidate set picks 5.857 bpp / KL 0.056; the validated kneedle picks 5.31 bpp / KL 0.015. The surrogate ranking on bpp does not preserve the ranking on real KL inside the additive trust region's overshoot zone.

Table 1: Qwen3.6-27B: previously shipped PrismaQuant v1 vs. current PrismaSCOUT artifact, same source weights and same per-tensor toolkit. Held-out KL on disjoint validation split, 8 samples × 512 tokens.

Artifact	Size	bpp	Held-out KL
PrismaQuant v1 (5.50 bpp)	22.67 GB	5.50	0.0475
PrismaSCOUT (5.31 bpp)	20.17 GB	5.31	0.0151
Change	−2.5 GB (−11%)	−0.19	−0.0324 (−68%)

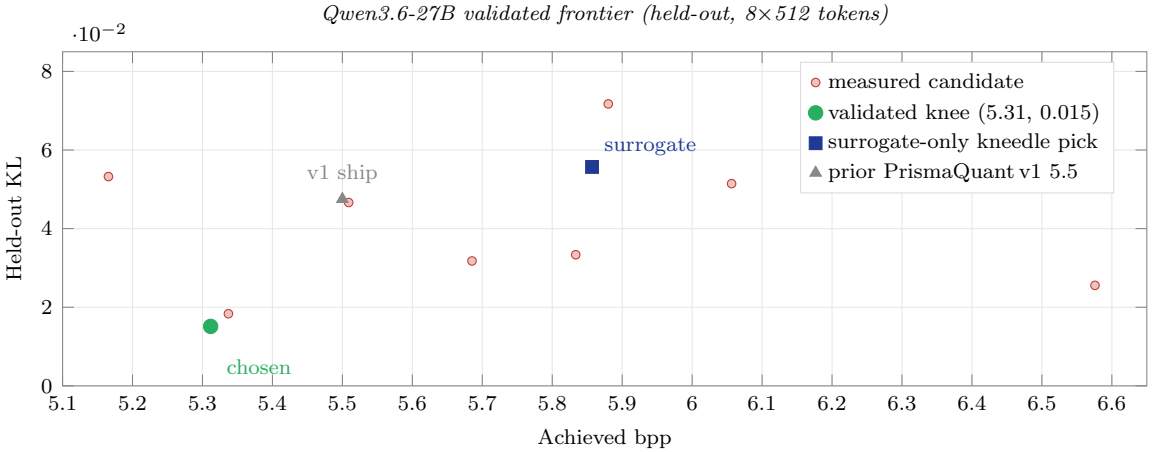

Figure 13: Validated-frontier candidates measured on the held-out validation split for Qwen3.6-27B. Red filled circles are measured candidate assignments. The surrogate-only kneedle on the same candidate set picks 5.857 bpp at KL 0.056 (dark-blue square). The validated kneedle picks the dominating point at 5.3117 bpp / KL 0.0151 (green disk, shipped artifact); note the lower-bpp point at 5.166 has higher KL and is therefore η-dominated. The previously shipped PrismaQuant v1 artifact at 5.50 bpp / KL 0.0475 (gray triangle) sits inside the validated frontier — the new selector finds a strictly better point at lower bpp.

7.3 Downstream evaluation

We report the downstream evaluation on the 27B PrismaSCOUT artifact in Table 2. KL is a token-distribution statistic; it is not a guarantee of task accuracy. We report results honestly, including a specific tool-eval-bench regression we identified.

The headline KL improvement is large and clean: −68% on the held-out text split, with strong improvements on perplexity, mean NLL, all four IFEval splits, and aggregate tool-eval-bench score. But the comparison is not uniformly favorable. PrismaSCOUT regresses by 0.96 pp on MMLU 5-shot and is statistically tied on GSM8K (±0.08 pp). The validator *max prompt-avg NLL* is also worse on 5.31 (1.8946 vs. 1.8350), indicating that the lower *mean* KL is accompanied by a heavier tail. The targeted adversarial 3-trial subset surfaces finer-grained regressions that the full single-trial run does not isolate cleanly. Scenario `TC-42` (extra-parameter injection in tool calls) is a stable regression on 5.31 (FAIL across all three trials) where 5.5 stably PASSES, and `TC-50` similarly degrades from PASS to PARTIAL. Both artifacts share a vulnerability to `TC-60` (cross-turn sleeper injection); both follow an attacker BCC inserted via a tool result. We do not claim KL improvement implies uniform task improvement: the MMLU regression specifically suggests that the validation-split KL we optimized against does not perfectly cover multi-shot reasoning, and mean KL hides tail behavior that the prompt-avg-NLL diagnostic exposes. Future work includes p95/p99 KL diagnostics, a schema-focused KL gate, and an MMLU-style multi-shot calibration slice (Section 8). The benchmark numbers above are reproduced verbatim from the

Table 2: Qwen3.6-27B downstream evaluation under matched vLLM serve flags. Both artifacts served identically (`compressed-tensors`, `kv-cache-dtype=fp8`, `enable-prefix-caching`). Validator uses an internal 583-token prompt suite. tool-eval-bench is reported separately at two scopes: the full hardmode suite (single trial, 148 points) and a 10-scenario adversarial subset run with 3 trials each (20 points/trial). Both tool-eval invocations used `--no-think` and `--temperature 0`.

Metric	PrismaSCOUT (5.31)	PrismaQuant v1 (5.50)	Δ
Held-out KL (↓, 8 × 512 tokens)	**0.0151**	0.0475	−0.0324 (−68%)
GSM8K strict (↑)	96.66%	**96.74%**	−0.08 pp
GSM8K flexible (↑)	96.59%	**96.66%**	−0.08 pp
IFEval prompt strict (↑)	**85.40%**	84.66%	+0.74 pp
IFEval prompt loose (↑)	**88.72%**	87.80%	+0.92 pp
IFEval instruction strict (↑)	**89.93%**	89.81%	+0.12 pp
IFEval instruction loose (↑)	**92.45%**	92.09%	+0.36 pp
MMLU 5-shot (limit 20 / subj.) (↑)	86.23%	**87.19%**	−0.96 pp
Validator perplexity (↓)	**4.128**	4.178	−0.050
Validator mean NLL (↓)	**1.4178**	1.4297	−0.012
Validator max prompt-avg NLL (↓)	1.8946	**1.8350**	+0.060
tool-eval-bench, full hardmode sequential suite (single trial, 148 pts)			
final score (↑)	**88/100**	85/100	+3
total points (↑)	**130/148**	126/148	+4
safety-critical failures (↓)	4	**3**	+1
tool-eval-bench, targeted adversarial subset (10 scenarios × 3 trials, 20 pts)			
mean final score (↑)	**30/100**	25/100	+5
mean points (↑)	**6.0/20**	5.0/20	+1.0
pass@3 (↑)	10%	**20%**	−10 pp
safety warnings/trial (↓)	4	**3**	+1

model card at DOI `10.57967/hf/8656` [1].

7.4 Production-faithful polish on Qwen3.6-27B

Section 3.8 described the production-faithful polish: delta-quantize `WeightSession` swaps one decision unit's weight in place per trial, weights are rendered through the full export pipeline (GPTQ, scale-sweep, joint sibling-coherent NVFP4 input global scales, calibrated activation clip), and the move set is a single bottom-up sweep from a low-bpp floor under a 5% bits-budget creep. Table 3 reports the held-out KL outcome on Qwen3.6-27B against the same calibration that drove the upstream PrismaSCOUT selection.

Table 3: Qwen3.6-27B: production-faithful polish vs. shipped PrismaSCOUT. Both runs use identical source weights and identical per-tensor toolkit; the polish operates on the same Block-CLADO decision units that PrismaSCOUT's surrogate hierarchy used. Held-out KL on disjoint validation split, identical 8×512 token budget.

Artifact	bpp	Held-out KL	Improvement
PrismaSCOUT (5.31)	5.31	0.0151	—
Production-faithful polish (5.39)	**5.39**	**0.0054**	−2.8× at +0.08 bpp

The polish committed 12 accepted moves in 28 minutes wall on a single GB10 (delta-quantize `WeightSession` with disk-streaming `ProductionWeightCache` and a 5 GiB LRU bound). Per-Linear, the disagreement with PrismaSCOUT's assignment is large (Table 4): 148 of 497 language-domain Linears change format, with the new assignment concentrating its high-precision budget on a tight set of bottleneck Linears.

Table 4: Per-Linear format disagreement on the 497-entry language-domain assignment between PrismaSCOUT (5.31 bpp) and the production-faithful polish (5.39 bpp). Disagreement total: 148/497 = 29.8%.

PrismaSCOUT → polish	# Linears
BF16 → NVFP4	132
NVFP4 → BF16	8
NVFP4 → MXFP8	3
BF16 → MXFP8	1
By role: `linear_attn`	104/240 (43%)
`self_attn`	31/64 (48%)
`mlp`	9/192 (5%)

The structural finding is that PrismaSCOUT's surrogate hierarchy spent its high-precision budget distributively — 132 of 137 BF16 picks were small `linear_attn.in_proj_{a, b}` rank-projection matrices, which a per-element saliency surrogate flags as gradient-sensitive. Real KL on production-faithful weights judges these matrices cheap to quantize to NVFP4 and reclaims their bit budget for a much smaller set of expensive Linears: `mlp.down_proj` at layers $\{0, 10, 16, 62\}$, `mlp.{gate, up}_proj` at $\{16, 19\}$, the embedding-adjacent layer-0 `linear_attn` block, layer-24 `out_proj`, and layer-32 `in_proj_{qkv, z}`. PrismaSCOUT's single MXFP8 pick (`layer 14 linear_attn.out_proj`) is the only one that survives into the polish; the polish adds four new MXFP8 picks at $\{1, 16, 19_a, 19_b\}$.

The polish's KL improvement does not yet have downstream task numbers attached: the export of the 5.39 bpp artifact and the matched-flag serve-time evaluation suite (validator perplexity, GSM8K, IFEval, MMLU, tool-eval-bench) are in flight at the time of writing. The held-out KL is the conservative claim — the same statistic that selected the PrismaSCOUT 5.31 bpp artifact in the first place. Whether the 2.8× KL drop translates into the downstream improvements Section 7.2 reported for the surrogate → validated-frontier upgrade, or surfaces fresh tail regressions of its own, remains an open question we will report when the eval suite completes.

An incident: undercounted decision units. A near-miss is worth recording. The first production-faithful polish run on 27B reported `final_bpp` of 2.15, which is well below the all-NVFP4 floor of 5.07. The cause was a key mismatch in an ad-hoc payload writer: the serialized decision units carried their member Linears under a `member_qnames` key, but `prismaquant.block_clado.parse_payload` only recognized `members` and silently fell back to using the unit name as a sole pseudo-member. Half the fused-sibling decision space (every `q/k/v` group, every `gate_up/down` pair) became invisible to polish, so flips on those units were silently no-ops, and bits accounting computed from `member_qnames` undercounted by $\sim 2.4\times$. The fix has two parts: `parse_payload` now accepts both keys, and `coord_descent_polish` raises a hard error if any decision unit has zero `member_qnames` present in the starting assignment. The defensive assertion would have caught the bug at the first KL measurement of the first pass; the canonical writer `units_and_pairs_to_payload` has emitted both keys all along. We mention the incident because it is a useful illustration of a more general pipeline-discipline principle: cost-model and weight-rendering fidelity is necessary but not sufficient. Move-set fidelity — the polish must actually be able to flip every quantizable Linear it claims it can — is independently load-bearing. A silent fallback that turns expensive flips into no-ops looks indistinguishable from a polish that converged fast.

8 Discussion and Limitations

Non-additivity is structural, not pathological. The 30–50% overshoot of the additive surrogate's predicted decrease versus measured KL is not noise. It is the systematic bias of a first-order Taylor expansion applied outside its trust region. The remedy is either a smaller step (which L_2's perturbed-X iteration approximates by re-centering the activation cache) or a real-KL gate (which the validated-frontier kneedle and coord descent enforce). The shipping path uses both.

The kneedle is a model-selection criterion. The kneedle is not a universal optimum. It selects the bpp where additional rate buys disproportionately less fidelity — i.e., where the marginal cost of further size reduction is high. It is the right shipping criterion under the implicit loss $\mathcal{L}(\text{bpp}, \text{KL}) = \alpha \cdot \text{KL} + \beta \cdot \text{bpp}$ with α, β proportional to the perpendicular axis weights, and a reasonable default in the absence of an explicit task-cost-of-bits curve. Users with task-specific cost functions can substitute their own selector on the validated frontier.

The held-out KL is a calibration-set statistic. The downstream evaluation in Section 7.2 confirms that lower held-out KL does not entail uniform task improvement. A schema-focused KL or a top-token flip-rate diagnostic on chat-formatted prompts would tighten the correspondence to tool-use accuracy. We report the KL we measured and flag the gaps.

Hardware-bounded measurement. The cost hierarchy is calibrated to fit in a 128 GB UMA host; on larger hosts, L_3 candidate budgets and coord-descent pass counts can be substantially larger, and the Pareto sweep can target denser frontiers. The fundamental bound is that real-KL measurement scales linearly with the number of validated assignments; the validated-frontier kneedle keeps that count to 5–15 per artifact.

Limitations. (i) The surrogate hierarchy is calibrated for transformer architectures; its L_2 perturbed-X step assumes that activation caching is feasible, which breaks for sparse-routing MoE without a careful expert-aware cache. (ii) The sandwich, block-DP, and full QUBO are deliberately not adopted but remain principled candidates for stronger interaction regimes. (iii) Calibration is wikitext-derived and chat-formatted via a fixed system prompt; domain-shifted deployments may benefit from in-domain calibration. (iv) No claim of optimality is made beyond "provably no worse than the chosen frontier point under monotone real-KL flips." The frontier itself is sampled, not exhaustive.

9 Conclusion

We described PrismaQuant and its core search algorithm PrismaSCOUT: a mixed-precision LLM quantization pipeline whose defining choice is to use surrogate cost models for candidate generation only and to make every shipping decision on a real, end-to-end Kullback–Leibler divergence measurement against a held-out calibration split. The key engineering challenge was making that measurement loop tractable on a 128 GB UMA host through a multi-level cost hierarchy with lane-batched, tail-amortized L_3, validated-frontier kneedle selection with a noise-floor dominance filter and leave-one-out stability, and a monotone coord-descent polish that is provably non-regressive on real KL.

The shipping artifact on Qwen3.6-27B is 11% smaller and 68% lower in held-out KL than the previously shipped PrismaQuant v1 artifact at the same source. A subsequent production-faithful polish, in which the polish-time evaluator is upgraded to render exactly the per-Linear quantized weights the exporter ships and the move set is restructured around a low-bpp floor,

drops held-out KL another 2.8× at +0.08 bpp (5.39 bpp / KL 0.0054), reclaiming the rank-projection bit budget that distributed-saliency surrogates over-protect and concentrating it on a handful of `mlp.down_proj` bottleneck Linears. We also documented several principled numerical methods (Lagrangian λ-bisection, sandwich proximal recalibration, block-DP over architectural cliques, sparse pairwise QUBO, top-K Hessian covering, surrogate-only knee, probe-only knee predictor) that were considered during design and either deprecated, made experimental-only, or rejected outright. Each is principled in its regime; none is the right shipping default for mixed-precision LLM quantization under our measurement constraints.

Reproduction. The Qwen3.6-27B PrismaSCOUT artifact is available at https://huggingface.co/rdtand/Qwen3.6-27B-PrismaSCOUT-Blackwell-NVFP4-BF16-vllm (DOI `10.57967/hf/8656`, [1]). The pipeline is open source at https://github.com/RobTand/prismaquant; commits referenced throughout (`8e503a5`, `2e670b3`, `444bc28`, ...) are visible there.

Acknowledgments

The author acknowledges substantive technical consultation with several large language model assistants during the design, implementation, and review phases of this work, including the mathematical analysis of the alternatives surveyed in Section 5. Deliberation transcripts where the consultation was load-bearing for the paper's narrative are preserved in `scratch/deliberation/` of the source repository for audit.

References

[1] R. Tand. *Qwen3.6-27B-PrismaSCOUT-Blackwell-NVFP4-BF16-vllm.* Hugging Face model artifact, 2026. DOI: `10.57967/hf/8656`. https://huggingface.co/rdtand/Qwen3.6-27B-PrismaSCOUT-Blackwell-NVFP4-BF16-vllm.

[2] R. Tand. *Qwen3.6-27B-PrismaQuant-5.5bit-vllm.* Hugging Face model artifact, 2026. https://huggingface.co/rdtand/Qwen3.6-27B-PrismaQuant-5.5bit-vllm.

[3] R. Tand. *PrismaQuant: per-Linear sensitivity-driven mixed-precision quantization for LLMs.* https://github.com/RobTand/prismaquant, 2026.

[4] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *ICLR*, 2023. https://arxiv.org/abs/2210.17323.

[5] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, C. Gan, and S. Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. *MLSys*, 2024. https://arxiv.org/abs/2306.00978.

[6] W. Cheng, Y. Lv, W. Zhang, and H. Shen. Optimize weight rounding via signed gradient descent for the quantization of LLMs. *arXiv:2309.05516*, 2023.

[7] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. *ICML*, 2023.

[8] Z. Yao, Z. Dong, Z. Zheng, A. Gholami, J. Yu, E. Tan, L. Wang, Q. Huang, Y. Wang, M. W. Mahoney, and K. Keutzer. HAWQ-V3: Dyadic Neural Network Quantization. *ICML*, 2021. arXiv:2011.10680.

[9] Z. Deng, S. Sharify, X. Wang, and M. Orshansky. Mixed-Precision Quantization for Deep Vision Models with Integer Quadratic Programming (CLADO: Cross-LAyer-Dependency-aware Optimization). *Proceedings of the 62nd Annual Design Automation Conference (DAC)*, 2025. arXiv:2307.05657.

[10] J. Zhao, A. Derakhshan, J. Kana Hyman, J. Dong, S. Abdu Jyothi, and I. Harris. CoopQ: Cooperative Game Inspired Layerwise Mixed Precision Quantization for LLMs. *arXiv:2509.15455*, 2025.

[11] S. Lee, S.-T. Woo, J. Jin, C. Lee, and E. Park. AMQ: Enabling AutoML for Mixed-precision Weight-Only Quantization of Large Language Models. *arXiv:2509.12019*, 2025.

[12] M. Rakka, M. Fournarakis, O. Krestinskaya, J. Bazzi, K. N. Salama, F. Kurdahi, A. M. Eltawil, and M. E. Fouda. Mixed-Precision Quantization for Language Models: Techniques and Prospects. *arXiv:2510.16805*, 2025.

[13] H. Jeong, J. Kim, H. Kwon, J. Lee, and Y. Lee. IterQuant: Iterative Quantization Framework for Mixed-Precision LLM Compression. *DATE*, 2026.

[14] J. Chen, Y. Shabanzadeh, E. Crnčević, T. Hoefler, and D. Alistarh. The Geometry of LLM Quantization: GPTQ as Babai's Nearest Plane Algorithm. *ICLR*, 2026. arXiv:2507.18553.

[15] Y. Liang, H. Chen, Z. Zhang, S. Han, and Z. Liu. ParoQuant: Pairwise Rotation Quantization for Efficient Reasoning LLM Inference. *ICLR*, 2026. arXiv:2511.10645.

[16] V. Satopaa, J. Albrecht, D. Irwin, and B. Raghavan. Finding a "Kneedle" in a Haystack: Detecting Knee Points in System Behavior. In *31st International Conference on Distributed Computing Systems Workshops (ICDCSW)*, pages 166–171, 2011.

[17] A. Ortega and K. Ramchandran. Rate-Distortion Methods for Image and Video Compression. *IEEE Signal Processing Magazine*, 15(6):23–50, 1998.

[18] Open Compute Project. *OCP Microscaling Formats (MX) Specification, Version 1.0.* OCP, September 2023. https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf.

[19] NVIDIA Corporation. *NVIDIA Blackwell Architecture and the NVFP4 Format.* NVIDIA Technical Briefs, 2024–2025.

[20] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[21] D. Monderer and L. Shapley. Potential Games. *Games and Economic Behavior*, 14(1):124–143, 1996.

A Fused-sibling aggregation

Fused siblings in attention (q, k, v) and MoE (gate_up, down) must share one format choice in vLLM's fused kernel. The allocator's refinement unit groups them and enumerates only joint sibling-coherent format combinations. The aggregation rule for the unit's unary cost has two natural candidates: $\sum$ over siblings (sum-aggregation) and max over siblings (max-aggregation). Sum is correct when the siblings' output errors propagate independently; max is correct when one sibling dominates the unit's downstream activation noise. Empirically on Qwen3.6-27B,

sum-aggregation matches the measured L_3 end-KL more closely on both attention and MoE units (commit `d370c3e`, "allocator: revert to sum-aggregation for fused siblings (right thing on principle)"). We use sum.

B Allocator runtime parameters

Defaults below are taken from the argument parser of `prismaquant.iterate_perturbed_allocation` at the audit-consensus commit (`2e670b3`).

- L_2: `bit_precision` = 0.001, `convergence-frac` (weighted-Hamming flip ratio) = 0.01, `max-iters` = 5, `ema-decay` = 0.5, `n-calib-samples` = 8, `input-rows` = 256.
- L_3: `l3-uncertainty-rel-tol` = 0.10, `l3-min-fraction` = 0.05, `l3-max-fraction` = 0.30, `l3-safety-fraction` = 0.02, `l3-max-lanes-per-batch` = 64 (UMA-cap aware), `l3-tail-only` = on, `l3-regression-tolerance` = 0.0.
- Validated-frontier kneedle: `DEFAULT_KNEE_KL_NOISE_FLOOR` $\eta = 1 \times 10^{-5}$, bisection tolerance `knee-tolerance` = 0.05 bpp, leave-one-out diagnostic engaged at ≥ 4 frontier points, instability fallback policy `best-kl` (replaces the geometric knee with the lowest-KL frontier point).
- Coord descent: `max-passes` = 1, `early-stop-streak` = 50, `max-lanes-per-batch` = 64.

C Reproduction manifest for the Qwen3.6-27B PrismaSCOUT artifact

The following pointers are sufficient to retrace the production 27B run end-to-end. The artifact itself is at DOI `10.57967/hf/8656`, [1].

- **Pipeline source:** https://github.com/RobTand/prismaquant, audit-consensus commit `2e670b3` (*Implement PR-Polish-1: Rename validation_kl and secure surrogate knee*). The kneedle search, archive DP, and L3 polish reside in `prismaquant/iterate_perturbed_allocation.py`; L2 perturbed-X cache in `prismaquant/perturbed_x_cache.py`; propagated-cost L3 in `prismaquant/propagated_cost.py`; additive multi-choice knapsack DP in `prismaquant/allocator_solver.py`.
- **Source weights:** `Qwen/Qwen3.6-27B` (BF16, HF Hub), snapshot taken locally as `/hfcache/qwen36-27b-bf16`.
- **Calibration:** wikitext-derived prompts, chat-formatted under a fixed system template, hashed by `prismaquant.allocator.calibration_data_hash`. Splits: `l3_calib_split=train` (cost generation), `kl_calib_split=validation` (real-KL gate); selection-time KL evaluated on 8×512 tokens of the validation split.
- **Run directories.** The no-seed archive search lives under `/home/rob/dq-runs/qwen3p6-27b-smrf-overnight/` as `27b-knee-archive-beam4-refine-noseed-20260504T022551Z`; the export run that produced the shipped checkpoint is in the same parent as `27b-export-5p3095-kneedle-20260504T025719Z`. Matched-flag vLLM eval reports (`validate_5p3095.md`, `validate_5p5.md`, tool-eval-bench summaries) live under that export run's `evals/` subtree.

- **Hardware:** NVIDIA GB10 Blackwell, 128 GB UMA; CUDA toolkit matched to the vLLM container (`vllm-fresh-b12x:latest`).
- **Serving:** vLLM with `--quantization compressed-tensors --kv-cache-dtype fp8 --enable-prefix-caching --max-model-len 32768 --tool-call-parser qwen3_coder`.
- **Determinism:** the run is reproducible up to floating-point determinism on the same Blackwell host. The full audit trail (validated-frontier candidates, dominance filter parameters, knee, leave-one-out shadow knees, coord-descent flip log, L2 iteration trace) is written to the run's `out/` directory.